Universidad ORT Uruguay

Facultad de Ingeniería

VirtualPark: Parque Temático Virtual Interactivo

Nicolás Piriz - 310896, Santiago Suárez - 313578, Manuel Stapff - 303636

Repositorio GitHub: https://github.com/IngSoft-DA2/310896-313578-303636

Tutor: Francisco Bouza, Bruno Acosta, Sofia Duclos

2025

Índice

Índice 2
Descripción general del trabajo y del sistema 3
Justificación de la estructura de namespaces 4
Diagramas de interacción 7
Jerarquías de Herencia y Patrones 7
Modelo de tablas de la estructura de la base de datos 8
Explicación de los mecanismos generales y principales decisiones de diseño tomadas 8
Objetivo del diseño 8
Principales mecanismos implementados 8
Arquitectura por Capas 8
Principios de Diseño de Paquetes (Arquitectura) 9
Inversión de Dependencias (DIP) y Fabricación Pura (GRASP) 11
Principio de Segregación de Interfaces (ISP) 12
Patrón Repositorio y Variaciones Protegidas (GRASP) 12
Patrón Facade (Fachada) 13
Patrón Strategy y Principio Abierto/Cerrado (OCP) 13
Mecanismo de Acceso a Datos 14
Descripción del Manejo de Excepciones 14
Mecanismo de Fecha y Hora Centralizado (Abstracción del Reloj) 15
Estrategia de Herencia para Usuarios y Visitantes (TPH) 15
Autenticación y Autorización 16
Resumen del Mecanismo 16
Flujo 1: Autenticación (Login) 16
Flujo 2: Autorización (Acceso a Endpoint Protegido) 17
Filtros de Acción para Lógica Transversal (Autorización) 18
1. Resumen de las Principales Mejoras al Diseño 18
2.Informe de Métricas de Diseño: 20
2.1 Metodología de análisis 20
Descripción del flujo en el frontend de VirtualPark-App 26
Estructura, Componentes y Signals 26
Enrutamiento y protección de vistas 26
Servicios, lógica de negocio e interceptor 27
Credenciales, roles y localStorage 27
Interfaces y tipado estricto 27
Identidad visual: integración de Tailwind CSS y Angular Material 28

Carga dinámica de estrategias de puntaje..... 28
Interfaz para Desarrolladores Externos.....28
Descripción de la Interfaz.....29
Guía para Crear una Nueva Estrategia.....29
Paso 1: Crear un Proyecto de Librería de Clases.....29
Paso 2: Agregar Referencias Requeridas.....29
Paso 3: Implementar IScoringStrategy.....29
Paso 4: Compilar el Plugin..... 30
Paso 5: Desplegar el Plugin..... 30
Paso 6: Activar la Estrategia..... 30
Mecanismo de Descubrimiento por Reflection.....30
Clase ScoringStrategyLoader..... 30
Proceso de Carga..... 31
Integración con la Interfaz Gráfica..... 31
Diseño Preparado para Extensibilidad..... 31
Funcionalidad de Carga de Plugins desde UI..... 31
API REST para Gestión de Estrategias..... 31
Endpoints Disponibles.....32
Obtener Estrategias Disponibles.....32
Activar una Estrategia..... 32
Subir un Plugin..... 32
Ventajas del Diseño Implementado.....32
Justificaciones Propias.....32
Uso de IA..... 33
Errores o Bugs conocidos:..... 34
Especificación API..... 34
ANEXO..... 36
Diagrama de paquetes:.....36
Diagramas de interacción.....38
Diagramas de secuencia.....38
Diagrama de componentes..... 41
Diagrama de despliegue..... 42
Especificación API.....42
Diagramas de patrones..... 48
Informe de cobertura y análisis interpretativo de los resultados..... 50
Modelo de tablas de la estructura de la base de datos.....53
Video de demostración del uso de API..... 54
Guia de Instalacion:.....54

Descripción general del trabajo y del sistema

El proyecto consiste en el desarrollo de una plataforma de software para un parque temático virtual interactivo cuyo propósito principal es enriquecer la experiencia del visitante fusionando atracciones físicas con elementos digitales de gamificación. La aplicación “VirtualPark” fue desarrollada en c# utilizando el framework de .NET, utilizando la metodología de GitFlow. Considerando la arquitectura definida para el proyecto, se enfoca en el desarrollo del backend de la aplicación, el cual fue desarrollado siguiendo la metodología de desarrollo de **TDD** (Test Driven Development) con el framework MSTest, y en el frontend desarrollado en Angular 20.
El alcance de este trabajo incluye:

- La implementación de toda la lógica de negocio necesaria para soportar las funcionalidades descritas para los roles de Administrador, Operador y Visitante.
- El desarrollo de una API REST que exponga todas las operaciones requeridas por la aplicación.
- La implementación de un sistema de autenticación basado en tokens para proteger los endpoints privados.

Se utilizaron diversas tecnologías como Entity Framework, para manejar el tema de la persistencia en base de datos, Docker Desktop, para levantar un contenedor para la base de datos y DBeaver como cliente DBMS.

Se intentaron aplicar en todo momento las directrices y pautas de clean code, los principios SOLID y GRASP ponderando la legibilidad, el mantenimiento y el escalado, así como también los patrones de diseño vistos.

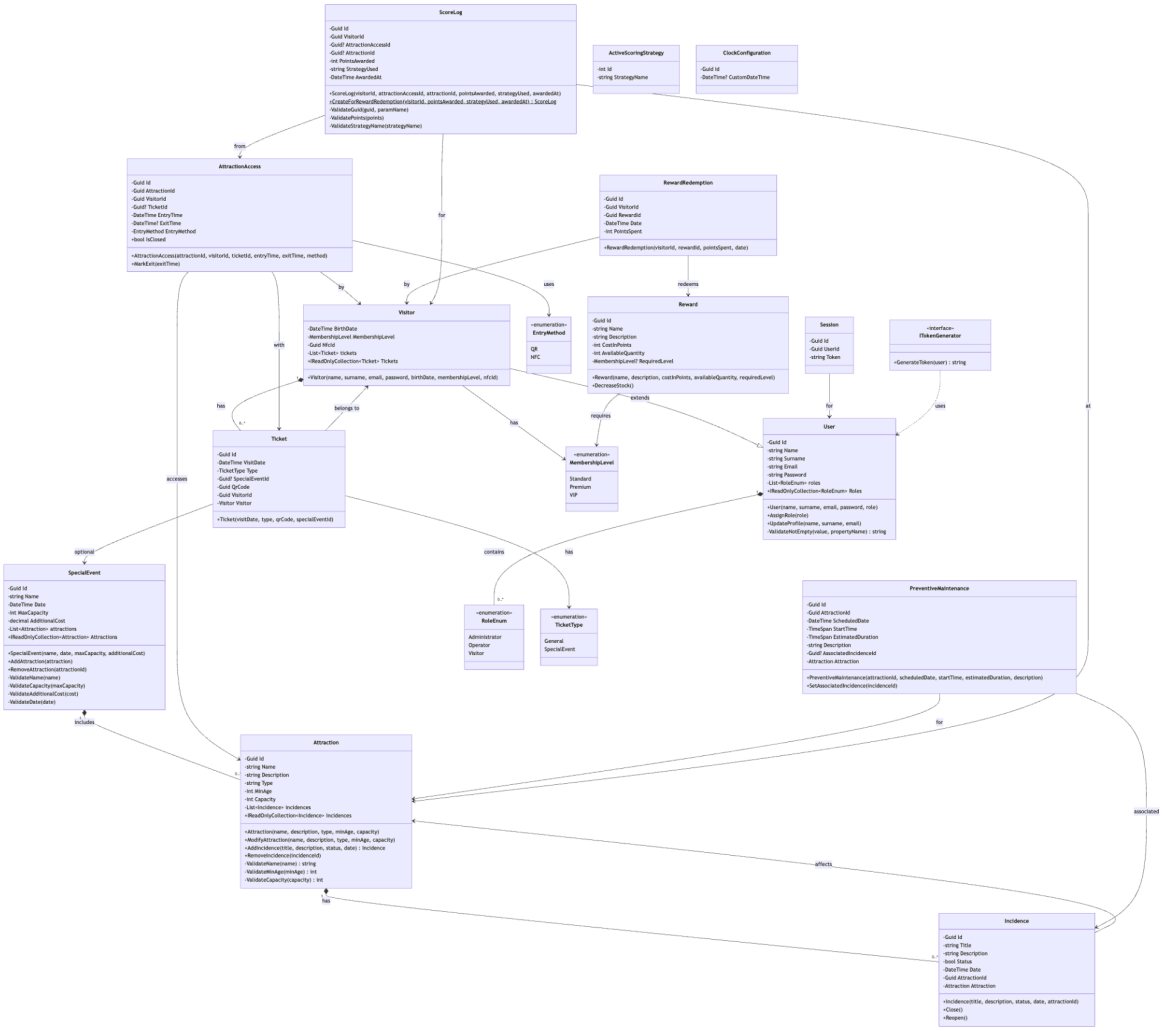
Justificación de la estructura de namespaces

Diagrama de paquetes

- VirtualPark.Domain:

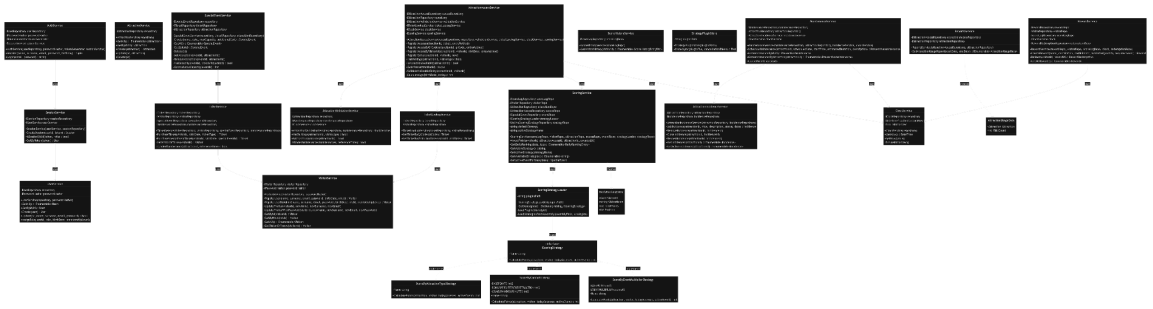
  Este paquete es el núcleo de la aplicación. Su responsabilidad es contener todas las entidades del negocio (User, Attraction, Ticket, etc.), los objetos de valor, los enums y las excepciones específicas del dominio. Define las reglas de negocio y las validaciones que son intrínsecas a los objetos, garantizando que estos siempre se encuentren en un estado consistente. No tiene dependencias de ningún otro proyecto de la solución.

Diagrama de clases de la capa del dominio:



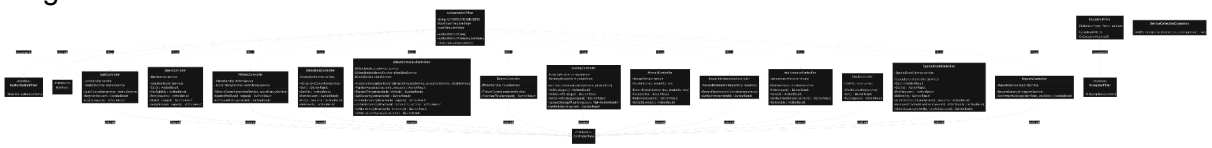
- VirtualPark.Application:

  Actúa como la capa de lógica de la aplicación, la Business Logic. Define y orquesta los casos de uso del sistema a través de servicios (ej. UserService, AttractionService, SpecialEventService, etc). Contiene las interfaces de los servicios y repositorios, y sus implementaciones coordinan las entidades del dominio para ejecutar las operaciones requeridas por la capa de presentación.
  Diagrama de clases de la capa de aplicación:



- VirtualPark.WebApi:

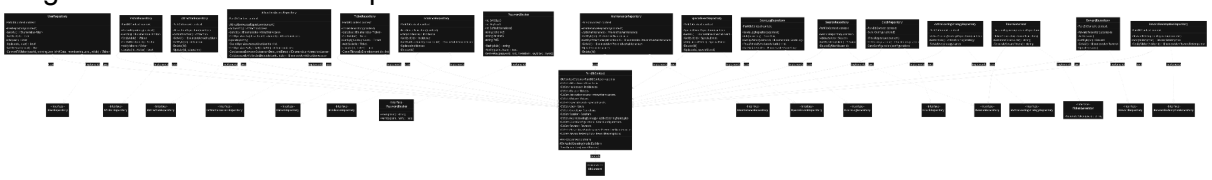
Es el punto de entrada y la capa de presentación de la solución. Su responsabilidad es exponer la funcionalidad del sistema a través de una API REST. Gestiona las peticiones y respuestas HTTP, la serialización/deserialización de datos a través de los DTOs, la configuración de rutas mediante controladores y endpoints y la implementación de mecanismos como la autenticación (AuthorizationFilter) y el manejo de errores (ExceptionFilter).
Diagrama de clases de la API:



- VirtualPark.Infrastructure:

  Es la capa de infraestructura que implementa los detalles técnicos de persistencia de datos y seguridad. Su responsabilidad es materializar las abstracciones definidas en el dominio mediante Entity Framework Core. Gestiona el acceso a la base de datos a través del ParkDbContext, define las configuraciones de mapeo objeto-relacional (ORM) y migraciones, implementa los repositorios concretos para cada entidad del dominio, y proporciona servicios de seguridad como el hasheo de contraseñas (PasswordHasher) y la generación de tokens de autenticación (TokenGenerator).

  Diagrama de clases de la capa de infraestructura:



- Proyectos de Pruebas (*.Tests):

  Cada proyecto principal de la solución tiene un proyecto de pruebas asociado. La única responsabilidad de estos paquetes es verificar el correcto funcionamiento del código de la capa correspondiente a través de pruebas unitarias y de integración, asegurando la calidad, robustez y mantenibilidad del software. Se implementa la librería Moq, para la aplicación de Mocks.

- VirtualPark.DTOs

  Es un proyecto el cual contiene todos los Dtos, tanto los de request como los de response. Se encarga de proveer los data transfer object para ocultar información que se va a devolver, y manejar las entradas para luego transformarlas en objetos de dominio. Además, contiene clases con métodos de mapping para facilitar esto.
  Diagrama de clases de la capa de DTOs

Dada la complejidad de la lectura de varios de estos diagramas de clases para los paquetes, se disponibilizan los mismos en el siguiente link de drive, bajo los nombres de

“Diagrama de Clases <paquete>.svg”:
https://drive.google.com/drive/folders/1kg4MtE54g7UJgIPlhqtHc6CXFiSTYplF?usp=sharing

Diagramas de interacción

Diagrama de secuencia de el registro de acceso a una atracción (RegisterAccess):

310896-313578-303636 Obligatorio 2
El diagrama muestra el flujo completo desde que un usuario intenta acceder a una atracción hasta que se le otorgan puntos por su visita.

Diagrama de secuencia de la compra de tickets para eventos especiales.

310896-313578-303636 Obligatorio 2

Esta es una operación crítica del sistema ya que los eventos especiales tienen capacidad limitada y es importante evitar vender más tickets de los permitidos.

Diagrama de secuencia del sistema de otorgamiento de puntos con diferentes estrategias de cálculo.

310896-313578-303636 Obligatorio 2

Diagrama de componentes:
Diagrama de Componentes

Jerarquías de Herencia y Patrones

Utilizamos una única herencia en el obligatorio que es la de User - Visitor.
Tenemos la clase user con sus atributos y una lista que maneja roles múltiples mediante una lista de RoleEnum, ya que los roles no son excluyentes.
Además, tenemos una clase visitor que hereda de user que además tiene los atributos de visitor específicos.
Adicionalmente tenemos clases para excepciones personalizadas, que heredan de Exception de .NET.
De la forma en que lo mapeamos y definimos estas herencias se aclarará más adelante.

Modelo de tablas de la estructura de la base de datos

310896-313578-303636 Obligatorio 2

Explicación de los mecanismos generales y principales decisiones de diseño tomadas

Objetivo del diseño

El diseño de la solución se ha centrado en la mantenibilidad, flexibilidad y escalabilidad, siguiendo las mejores prácticas vistas en clase y los principios de diseño de software. El objetivo fue crear una base de código que sea fácil de entender, modificar y extender, siempre cumpliendo con el objetivo de cumplir con la alta cohesión y el bajo acoplamiento. A continuación, se detallan las decisiones arquitectónicas y los patrones de diseño implementados.

Principales mecanismos implementados

Arquitectura por Capas

La solución se ha estructurado siguiendo una arquitectura por capas bien definida para separar las responsabilidades:

1. Capa de Presentación (Web API): Responsable de exponer los endpoints, recibir las peticiones HTTP, validar los datos de entrada (DTOs) y devolver las respuestas. No contiene lógica de negocio.
2. Capa de Lógica de Negocio (Dominio/Servicios): Contiene la lógica principal de la aplicación, las reglas de negocio y las entidades del dominio. Orquesta las operaciones interactuando con la capa de datos.
3. Capa de Acceso a Datos (Persistencia): Implementa el patrón Repositorio y se encarga de la interacción con la base de datos a través de Entity Framework Core.

Implementamos Dtos para así la API no expone entidades del dominio directamente y los cambios en el dominio no la afectan. Adicionalmente, permite evitar la sobreexposición de datos y la exposición no querida de datos sensibles.

Impacto en la Mantenibilidad: Esta separación estricta facilita la comprensión del sistema y el aislamiento de los cambios. Una modificación en las reglas de negocio no debería afectar a la API, y un cambio en la base de datos solo impacta en la capa de persistencia.

Principios de Diseño de Paquetes (Arquitectura)

La arquitectura de la solución está organizada en paquetes que siguen los principios de diseño de paquetes para asegurar la mantenibilidad y estabilidad.

- VirtualPark.Domain: Contiene solo las entidades y las excepciones del dominios. No tiene dependencias externas (excepto .NET). Es el paquete más estable.
- VirtualPark.Application: Contiene las interfaces de servicios (ej. IUserService) y la lógica de negocio pura (Servicios de Aplicación). Depende solo de Domain.
- VirtualPark.Infrastructure: Contiene implementaciones concretas de las interfaces de Application y Domain (ej. UserRepository, ParkDbContext). Depende de Application y Domain.

- VirtualPark.WebApi: Es el punto de entrada. Depende de Application (para invocar servicios) y DTOs.
- VirtualPark.DTOs: Contiene los Data Transfer Objects. Depende solo de Domain.

Este diseño sigue los siguientes principios:

- **Principio de Dependencias Acíclicas (ADP):** El grafo de dependencias es acíclico. Ya de por sí .NET no permite las dependencias entre proyectos de forma cíclica. Esto previene el "dll hell" y asegura que el proyecto se pueda compilar de forma predecible. Pero además, para nuestro diseño evitamos las dependencias cíclicas entre namespaces dentro del mismo proyecto de .NET, para cumplir con este principio.
- **Principio de Dependencias Estables (SDP):** Las dependencias fluyen en la dirección de la estabilidad.
    - VirtualPark.Domain es el más estable (dependido por todos, no depende de nadie).
    - VirtualPark.Application es muy estable (dependido por WebApi e Infrastructure).
    - VirtualPark.Infrastructure y VirtualPark.WebApi son inestables (dependen de otros, pero nadie depende de ellos).
    - Esta estructura es correcta. Los componentes volátiles (como la UI y la Base de Datos) dependen de los componentes estables (reglas de negocio y entidades).
- **Principio de Abstracciones Estables (SAP)**: Un paquete debe ser tan abstracto como estable.
    - VirtualPark.Application es el mejor ejemplo. Es un paquete muy estable (muchas dependencias entrantes) y es también muy abstracto (consiste casi enteramente en interfaces como IUserService, IRewardService, IAttractionAccessService, IUserRepository, etc.).
    - Esto permite que VirtualPark.Infrastructure (un paquete inestable) implemente estas interfaces, cumpliendo perfectamente con la Inversión de Dependencias (DIP) a nivel de arquitectura.
- **Principio de Cierre Común (CCP):** Las clases que cambian por las mismas razones deben estar en el mismo paquete.
    - Ejemplo: Se creó el paquete VirtualPark.Application.Maintenance. Si las reglas de negocio sobre cómo se programan o cancelan los mantenimientos cambian, todos los cambios están aislados en MaintenanceService.cs, dentro de ese único paquete.
    - Ejemplo: VirtualPark.Application.Scoring contiene ScoringService, RewardService y ScoreHistoryService. Todos están altamente cohesionados en torno al concepto de "puntos" y es probable que cambien juntos si la lógica de gamificación evoluciona.
- **Principio de Reutilización Común (CRP - Common Reuse Principle):** "Las clases en un paquete se reutilizan juntas. Si reutilizas una de las clases del paquete, las reutilizas todas."
    - **Análisis:** Este principio dicta que no debemos agrupar clases que no están relacionadas. Si un cliente depende de una clase en un paquete, debería depender de *la mayoría* de las clases en ese paquete.
    - **Aplicación:** VirtualPark.Domain es el mejor ejemplo. Las entidades (Attraction, Visitor, Ticket, ScoreLog) están intrínsecamente conectadas y se reutilizan como un conjunto cohesivo para definir el negocio. Al mantener el dominio cohesivo, se cumple este principio.
- **Principio de Equivalencia de Reutilización y Publicación (REP - Release Reuse Equivalency):** "La unidad de reutilización es la unidad de publicación."

- **Análisis:** Este principio se enfoca en la "publicación" (release) de software. Establece que los componentes reutilizables (como las librerías) deben ser publicados con un número de versión y un proceso de release formal, para que los consumidores puedan decidir cuándo actualizarse a una nueva versión.
- **Aplicación:** En esta arquitectura, este principio tiene una aplicabilidad limitada, no porque sea un mal diseño, sino por la naturaleza del proyecto. VirtualPark se desarrolla y despliega como una *única aplicación* (un solo sln). No estamos publicando VirtualPark.Domain como un paquete de versionado para que *otras soluciones* externas lo consuman. Si ese fuera el caso (por ej. si una app de "Ventas de Tickets" separada consumiera nuestro dominio), REP sería crítico.

Informe de Métricas de Diseño:

Metodología de análisis

Para evaluar la calidad del diseño del sistema se utilizó la herramienta NDepend aplicada sobre la solución, incluyendo los proyectos de backend y excluyendo los proyectos de pruebas automatizadas.

El análisis se realizó tomando como unidad principal los *assemblies* de la solución. Para cada *assembly* se extrajeron las siguientes métricas de diseño:

- **A (Abstractness)**: proporción de tipos abstractos (interfaces y clases abstractas) respecto al total de tipos del *assembly*.

- **I (Instability)**: mide qué tan inestable es un paquete en función de sus dependencias externas (I = Ce / (Ca + Ce)). Los valores cercanos a 0 indican paquetes más estables y los cercanos a 1, paquetes más inestables.

- **D (Distance to Main Sequence)**: distancia del punto (A, I) a la “línea ideal” definida por A + I = 1. Valores bajos de D indican que el paquete está mejor balanceado entre abstracción y estabilidad.

- **H (Relational Cohesion)**: mide el grado de cohesión interna de un paquete a partir de las relaciones entre sus tipos (referencias, llamadas a métodos, herencia, etc.). Valores más altos de H indican que las clases del paquete colaboran más entre sí y, por lo tanto, que el paquete está mejor cohesionado.

Estas métricas se utilizaron luego para analizar el cumplimiento de los principios de diseño a nivel de paquetes, tanto de **cohesión** como de **acoplamiento** y para fundamentar los cambios de diseño realizados entre la primera y la segunda entrega.

Resultados de métricas por assembly

En la tabla siguiente se resumen las métricas de diseño obtenidas con NDepend para los principales *assemblies* del backend del sistema. Para cada *assembly* se muestran el grado de abstracción (**A**), la inestabilidad (**I**), la distancia a la secuencia principal (**D**) y la cohesión relacional del paquete (**H**).

Estas métricas permiten caracterizar el rol de cada paquete dentro de la arquitectura y evaluar en qué medida se cumple el objetivo de mantener un dominio estable, bien abstraído y cohesivo, y capas de infraestructura y presentación relativamente más inestables.
Tabla

Análisis de métricas por assembly

VirtualPark.Domain

En el caso de *VirtualPark.Domain*, las métricas obtenidas fueron H = 0.86, A = 0.43, I = 0.25 y D = 0.23. Esto indica que el núcleo de dominio presenta un equilibrio adecuado entre tipos abstractos y concretos, con un 43 % de tipos abstractos (interfaces y clases abstractas). La inestabilidad de 0.25 muestra que se trata de un componente muy estable, con alto acoplamiento aferente (muchos otros paquetes dependen de él) y bajo acoplamiento eferente (pocas dependencias hacia afuera), lo cual es exactamente lo deseable para un módulo de dominio. La distancia a la secuencia principal (D = 0.23) lo sitúa muy cerca de la zona considerada ideal en el diagrama A–I, mientras que la cohesión relacional (H = 0.86) refleja que las clases del dominio están bien relacionadas entre sí.

VirtualPark.DTOs

En VirtualPark.DTOs las métricas medidas fueron H = 0.71, A = 0, I = 0.77 y D = 0.16. El valor de abstracción nulo (A = 0) indica que se trata de un paquete completamente concreto, lo cual es coherente con su naturaleza de conjunto de Data Transfer Objects: son clases simples destinadas a transportar datos entre capas, sin lógica de negocio ni jerarquías de interfaces. La inestabilidad relativamente alta (I = 0.77) es esperable, ya que estos objetos suelen estar expuestos a cambios derivados de la interacción entre capas y del contrato con clientes externos. La distancia a la secuencia principal (D = 0.16) es excelente y se encuentra dentro de la zona óptima del diagrama. La cohesión relacional (H = 0.71) se considera aceptable para este tipo de componente.

VirtualPark.DTOs muestra un excelente valor de distancia (D = 0.16), reflejando un equilibrio sano entre abstracción e inestabilidad. La falta de tipos abstractos es una decisión intencional y adecuada para clases de transferencia de datos. No se requiere refactorización, solo mantener la capa simple y concisa.

VirtualPark.Application

Para VirtualPark.Application se obtuvieron las métricas H = 0.88, A = 0.43, I = 0.86 y D = 0.20. El valor de abstracción (A = 0.43) pone de manifiesto una buena proporción de interfaces y servicios abstractos frente a sus implementaciones concretas, lo cual favorece la extensibilidad y permite desacoplar la lógica de orquestación del resto del sistema. La inestabilidad es elevada (I = 0.86), algo apropiado para una capa de aplicación que coordina casos de uso y depende tanto del dominio como de otras infraestructuras. La distancia a la secuencia principal (D = 0.20) se ubica justo en el límite superior de la zona considerada verde, y la cohesión relacional (H = 0.88) es buena, indicando que las clases dentro de este componente están fuertemente relacionadas entre sí en función de los servicios que ofrecen.

Como área de vigilancia, conviene monitorear que el valor de D no supere el umbral de 0.2, ya que un alejamiento mayor podría indicar un desbalance progresivo entre abstracción y estabilidad que convendría corregir con refactorizaciones puntuales.

VirtualPark.Infrastructure

En el caso de VirtualPark.Infrastructure, las métricas registradas fueron H = 1.05, A = 0, I = 0.99 y D = 0.01. La abstracción nula (A = 0) indica que se trata de un componente totalmente concreto, compuesto por implementaciones de repositorios, adaptadores de persistencia y otros servicios de infraestructura. La inestabilidad extrema (I = 0.99) es típica de esta capa, que depende de múltiples tecnologías externas y del modelo de dominio. La distancia a la secuencia principal (D = 0.01) es prácticamente perfecta, lo que sitúa a este paquete en una posición ideal dentro del diagrama A–I. No obstante, la cohesión relacional ligeramente superior a 1 (H = 1.05) sugiere una posible falta de cohesión interna, con clases que podrían estar asumiendo responsabilidades algo dispersas.

VirtualPark.WebApi

Por último, en VirtualPark.WebApi las métricas obtenidas fueron H = 1.27, A = 0, I = 1 y D = 0. La ausencia de abstracción (A = 0) confirma que se trata de un paquete compuesto exclusivamente por tipos concretos, principalmente controladores y elementos propios de la capa de presentación. La inestabilidad máxima (I = 1) es ideal para este tipo de componente, que debe depender de todos los demás sin que ninguno dependa de él. La distancia nula a la secuencia principal (D = 0) representa el valor perfecto en el diagrama, ubicando a WebApi exactamente sobre la línea A + I = 1. Sin embargo, la cohesión relacional elevada (H = 1.27) indica una baja cohesión interna, lo que sugiere que los controladores podrían estar asumiendo responsabilidades demasiado variadas o mezcladas.

Evaluación de principios de diseño a nivel de metricas

Principios de Cohesión:

En relación con el ***Release Reuse Equivalency Principle (REP)***, pasa lo mencionado anteriormente.

Respecto del **Common Closure Principle (CCP)**, el cumplimiento puede calificarse como medio-alto. Como mencionamos antes, las clases que cambian juntas están agrupadas en los mismos componentes. Domain (H = 0.86) muestra un buen agrupamiento de entidades y reglas de negocio, y Application (H = 0.88) evidencia servicios bien cohesionados en torno a casos de uso específicos. Sin embargo, Infrastructure (H = 1.05) mezcla responsabilidades relacionadas con persistencia, seguridad y migraciones en un mismo paquete, mientras que WebApi (H = 1.27) combina controladores, configuración y middleware.

En cuanto al **Common Reuse Principle (CRP)**, el sistema presenta un cumplimiento alto, como se mencionó anteriormente.

Principios de Acoplamiento:

El **Acyclic Dependencies Principle (ADP)** se cumple de forma perfecta.

El **Stable Dependencies Principle (SDP)** también se cumple de manera ejemplar. En todos los casos, los paquetes más inestables dependen de paquetes más estables. Por ejemplo, *Application* (I = 0.86) depende de *Domain* (I = 0.25), *Infrastructure* (I = 0.99) también depende de *Domain*, y *WebApi* (I = 1.0) depende tanto de *Application* (I = 0.86) como de *Infrastructure* (I = 0.99). En ninguna dependencia se observa que un paquete más estable dependa de uno más inestable. Esta dirección consistente de las dependencias contribuye a que los cambios se propaguen desde el núcleo más estable hacia las capas externas, y no al revés, alineándose con las buenas prácticas de arquitectura en capas.

Dependencia	I (origen)	I (destino)	✓/✗
Application → Domain	0.86	0.25	✓
Infrastructure → Domain	0.99	0.25	✓
WebApi → Application	1.0	0.86	✓
WebApi → Infrastructure	1.0	0.99	✓

Por último, el **Stable Abstractions Principle (SAP)** presenta un excelente cumplimiento. Los paquetes más estables tienden a ser más abstractos y los más inestables son predominantemente concretos. Domain, con A = 0.43 e I = 0.25, presenta un valor A + I = 0.68, cercano a la secuencia principal y adecuado para un núcleo de dominio que combina estabilidad con un nivel razonable de abstracción. Application (A = 0.43, I = 0.86) alcanza A + I = 1.29, reflejando una capa inestable pero fuertemente apoyada en interfaces y servicios abstractos. Infrastructure (A = 0, I = 0.99) y WebApi (A = 0, I = 1.0) son paquetes prácticamente totalmente concretos y muy inestables, lo que es coherente con su rol como capas de detalle tecnológico y presentación, respectivamente. En el diagrama de distancia, todos los componentes se encuentran en la zona verde/naranja, alejados tanto de la “zona de dolor” como de la “zona de inutilidad”, lo que confirma visualmente el buen balance global entre abstracción y estabilidad en la solución. Esto se puede visualizar en el “Abstractness versus Instability Diagram” provisto por NDepend.

Inversión de Dependencias (DIP) y Fabricación Pura (GRASP)

Utilizamos el mecanismo de inyección de dependencias para ayudarnos a implementar DIP (“*Los módulos de alto nivel no deben depender de módulos de bajo nivel: ambos deben depender de abstracciones.”*).

En lugar de que las clases creen sus propias dependencias, estas les son "inyectadas" por un contenedor de servicios. En lugar de que las clases generen sus propias dependencias, un contenedor de servicios se encarga de "inyectárselas". Esto se traduce que en vez de que las clases de alto nivel (ej. AttractionAccessService) instancien directamente sus dependencias (ej. new TicketLookupService()), estas dependen de abstracciones (interfaces como ITicketLookupService).

Dentro de este mismo tema, se abordó la creación de una clase la cual oficia como creador, cuyo objetivo principal es gestionar las inyecciones de dependencias. Esto es un claro ejemplo de una clase de fabricación pura, ya que no es parte del dominio, es un objeto

artificial para configuración, pero desacopla la creación de objetos de su uso y da un solo lugar para registrar dependencias.

- **Ventajas (Mantenibilidad y Modularidad):** Este enfoque desacopla los componentes. Permite intercambiar implementaciones (ej. sustituir UserRepository por MockUserRepository en pruebas) en un solo lugar (ServiceCollectionExtensions) sin afectar a los consumidores (controladores y servicios).
- **Consideraciones:** Si bien este patrón introduce una gestión de la complejidad en la configuración inicial y un leve acoplamiento al contenedor de DI del framework, los beneficios en modularidad, desacoplamiento y testeabilidad son fundamentales para la mantenibilidad a largo plazo.

Principio de Segregación de Interfaces (ISP)

El ISP se aplica para asegurar que los clientes (controladores, servicios) no dependan de métodos que no utilizan.

- **Implementación:** En lugar de crear interfaces "infladas", la arquitectura define interfaces cohesivas y específicas. Los controladores en WebApi dependen de estas interfaces segregadas, permitiendo que las implementaciones varíen sin afectar a los consumidores.
- **Ejemplo de Segregación:** Como se ve en el diagrama 4, un claro ejemplo de esto es la gestión de atracciones que se dividió funcionalmente en dos servicios distintos, cada uno con su propia interfaz:
    1. IAttractionService: Expone la funcionalidad de CRUD para la entidad Attraction.
    2. IAttractionIncidenceService: Expone la lógica de negocio para la gestión de incidencias (Crear, Cerrar, Reabrir).
- **Valor Aportado:** Un consumidor, como el AttractionAccessController, que solo necesita gestionar incidencias para validar la disponibilidad, puede depender exclusivamente de IAttractionIncidenceService, sin acoplarse innecesariamente a los métodos de CRUD de IAttractionService.

Patrón Repositorio y Variaciones Protegidas (GRASP)

Se ha implementado el patrón Repositorio para mediar entre la lógica de negocio y la capa de acceso a datos.

- **Implementación:** Cada agregado de dominio (ej. Attraction, Visitor, Ticket) define su propia interfaz de repositorio (ej. IAttractionRepository) en el paquete VirtualPark.Domain. Las implementaciones concretas (ej. AttractionRepository) residen en VirtualPark.Infrastructure y encapsulan la lógica de persistencia y las consultas de Entity Framework.
- **Decisión de Diseño (DIP y Variaciones Protegidas):** Esta abstracción es una aplicación directa del principio GRASP **Variaciones Protegidas**. La lógica de negocio (Application) está protegida contra cambios en la tecnología de persistencia. El sistema puede migrar de SQL Server a PostgreSQL modificando *únicamente* las

implementaciones en el paquete Infrastructure, sin generar impacto en los servicios de aplicación.
- **Impacto en la Mantenibilidad:** Centraliza la lógica de acceso a datos, evitando que las consultas de Entity Framework se dispersen por toda la aplicación. Si se necesita optimizar una consulta o cambiar la forma en que se accede a los datos de una entidad, solo se debe modificar su repositorio correspondiente. Esto respeta el **Principio de Responsabilidad Única (SRP)**.

Patrón Strategy y Principio Abierto/Cerrado (OCP)

El requerimiento de "Gamificación" especifica que el sistema debe soportar múltiples estrategias de puntuación que puedan ser intercambiables en tiempo de ejecución. Para resolver esto, se implementó el patrón **Strategy**, tal como se ve en el Diagrama 5

- **Implementación:**
  1. Se define una interfaz IScoringStrategy que define un único método: CalculatePoints(...).
  2. Se crean clases concretas que implementan esta interfaz para cada algoritmo (ej. ScoreByAttractionTypeStrategy, ScoreByComboStrategy).
  3. El servicio ScoringService (el 'Contexto' del patrón) mantiene una referencia a la estrategia activa (obtenida de IActiveScoringStrategyRepository) y la utiliza para calcular los puntos, delegando la ejecución del algoritmo específico.
- **Impacto en la Mantenibilidad:** Este patrón es una aplicación directa del **Principio Abierto/Cerrado (OCP)**. El ScoringService está *cerrado a modificaciones* (su lógica de orquestación no cambia), pero el sistema está *abierto a la extensión* (se pueden agregar nuevas estrategias de puntuación en el futuro simplemente creando nuevas clases que implementen IScoringStrategy y registrándolas).

Descripción de las principales funcionalidades y requerimientos de la letra

A continuación se detallan breves descripciones de la implementación de las funcionalidades principales que se detallan en la letra.

Mecanismo de Acceso a Datos

La persistencia de los datos se gestiona mediante Entity Framework (EF) Core 8.0, utilizando el enfoque Code First.

El modelo de la base de datos se define a través de clases de C# (entidades del dominio). EF Core se encarga de traducir este modelo a un esquema de base de datos relacional en SQL Server. Las interacciones con la base de datos (lecturas, escrituras, actualizaciones) se realizan a través de un DbContext, llamado “ParkDBContext” y utilizando LINQ para optimizar las consultas.

Gestión de Cambios: Las modificaciones en el esquema de la base de datos se gestionan a través de migraciones de EF Core, lo que permite versionar los cambios de la base de datos junto con el código fuente.

Descripción del Manejo de Excepciones

Se ha implementado una estrategia de manejo de excepciones centralizada para garantizar que la API siempre devuelva respuestas consistentes y seguras, sin exponer detalles internos de la implementación.

- Middleware de Excepciones: Se configuró un middleware en el pipeline, el cual está centralizado en un filtro ExceptionFilter. Este middleware intercepta cualquier excepción no controlada que ocurra durante el procesamiento de una petición.
- Flujo de Operación:
    1. Cuando ocurre una excepción, el middleware la captura.
    2. La excepción es registrada (logueada) para fines de depuración y auditoría.
    3. Se genera una respuesta HTTP estandarizada con un código de error genérico (ej. 500 Internal Server Error) y un mensaje amigable, evitando filtrar información sensible.
- Excepciones de Negocio: Para errores predecibles (ej. "Email duplicado", "Credenciales inválidas"), se utilizan excepciones personalizadas (InvalidCredentials , DuplicateEmailException). Estas son capturadas en la capa de la API (controladores) y mapeadas a códigos de estado HTTP específicos (404 Not Found, 400 Bad Request), proporcionando al cliente una respuesta más informativa.

Mecanismo de Fecha y Hora Centralizado (Abstracción del Reloj)

Para cumplir con el requerimiento de poder configurar la fecha y hora del sistema para facilitar las pruebas, se tomó la decisión de no usar DateTime.Now directamente en la aplicación.

- Implementación: Se creó una abstracción IClockService con una implementación (ClockService) registrada como Singleton, pero luego se cambió a scoped ya que DbContext no maneja bien los singleton.Igualmente, este servicio sigue funcionando como el responsable de proveer la fecha y hora actual al resto de la aplicación. Un endpoint específico en ClockController permite modificar el estado de este en tiempo de ejecución.
- Impacto en Mantenibilidad y Testeabilidad: Esta decisión es crucial. Permite que todo el sistema sea determinista en cuanto al tiempo. Se pueden escribir pruebas unitarias y de integración que simulen el paso del tiempo de forma controlada (ej. probar la validez de un ticket en una fecha futura) sin depender del reloj real del sistema, lo cual sería imposible de probar de manera automatizada.

Estrategia de Herencia para Usuarios y Visitantes (TPH)

El modelo de dominio distingue entre un User genérico y un Visitor, que es un tipo de usuario con atributos adicionales como fecha de nacimiento y membresía, el cual se comporta como User, así cumpliendo LSP. Para modelar esto en la base de datos, se eligió la estrategia de herencia Table-Per-Hierarchy (TPH) de Entity Framework Core.

- Implementación: Tanto la entidad User como Visitor se mapean a una única tabla Users en la base de datos. EF Core gestiona automáticamente una columna "Discriminator" para diferenciar qué tipo de usuario es cada fila.

- Impacto en Desempeño y Simplicidad: Utilizamos esta estrategia generalmente ofrece un mejor rendimiento para obtener datos polimórficos, ya que evita la necesidad de hacer JOINs complejos entre tablas. Simplifica el esquema de la base de datos, aunque a costa de tener columnas que pueden ser nulas para ciertos tipos de usuario. Fue una decisión de diseño consciente que priorizó el rendimiento de las consultas y la simplicidad del modelo.

Autenticación y Autorización

El mecanismo de seguridad es un sistema de **tokens opacos (stateful)** respaldado por base de datos, combinado con un filtro de acción personalizado para la autorización basada en roles.

Resumen del Mecanismo

A diferencia de un token JWT (stateless), donde la información del usuario (como sus roles) viaja dentro del propio token, este sistema utiliza un enfoque *stateful*:

1. **Token Opaco:** El token generado no es más que un string aleatorio y criptográficamente seguro generado en la clase TokenGenerator.cs. No contiene datos del usuario. Este token no tiene fecha de expiración y se actualiza cada vez que se hace el login.
2. **Persistencia de Sesión:** Este token actúa como una key que se almacena en la tabla Sessions de la base de datos. Esta tabla mapea el token con el UserId al que pertenece.
3. **Autorización Centralizada:** Un filtro personalizado, del cual hablaremos más adelante, intercepta *todas* las peticiones a endpoints protegidos, valida el token contra la base de datos y verifica los roles del usuario.

Flujo 1: Autenticación (Login)

El objetivo es validar las credenciales del usuario y entregarle un token de sesión.

1. Un usuario (Admin, Operador o Visitante) envía su email y password al endpoint POST /api/Auth/login.
2. El AuthController (Controlador GRASP) recibe el LoginRequestDto y delega inmediatamente a IAuthService.Login().
3. AuthService (Application/Users/AuthService.cs) orquesta el flujo: a. Llama a IUserRepository.GetByEmail(email). (Gracias a TPH, esto funciona para User y Visitor). b. Si no existe, lanza InvalidCredentialsException. c. Llama a IPasswordHasher.Verify() para comparar la contraseña enviada con el hash almacenado en la BD. d. Si no coincide, lanza InvalidCredentialsException. e. Si las credenciales son válidas, llama a ITokenGenerator.GenerateToken(user).
4. TokenGenerator genera un string aleatorio seguro (32 bytes, codificado en Base64). b. Llama a ISessionRepository.Add() para crear una nueva fila en la tabla Sessions que vincula este token con el user.Id.
5. El AuthService recibe el token de vuelta y lo retorna al controlador, que lo envía al cliente en un AuthResultDto. Este token no tiene fecha de expiración y se actualiza cada vez que se hace el login.

Flujo 2: Autorización (Acceso a Endpoint Protegido)

El objetivo es validar el token enviado por el cliente y verificar si tiene permiso para acceder al recurso.

1. El cliente realiza una petición a un endpoint protegido (ej. GET /api/Attractions), incluyendo el token en el encabezado: Authorization: Bearer <token_string>.
2. El filtro [AuthorizationFilter] (WebApi/Filters/AuthorizationFilter.cs), que decora el controlador, intercepta la petición antes de que llegue al método (Get()).
3. AuthorizationFilter.OnAuthorization:
    a. Extrae el token del encabezado.
    b. Obtiene ISessionService por Inyección de Dependencias.
    c. Llama a ISessionService.GetByToken(token).
4. SessionService (Application/Session/SessionService.cs):
    a. Llama a ISessionRepository.GetByToken(token) para buscar el token en la tabla Sessions.
    b. Si no lo encuentra, lanza KeyNotFoundException.
    c. Si lo encuentra, obtiene el UserId de la sesión.
    d. Llama a IUserService.GetById(userId).
    e. UserService llama a IUserRepository.GetById(userId), que (gracias a TPH) retorna la entidad User o Visitor completa.
    f. SessionService retorna el objeto User (con su lista de Roles) al filtro.
5. De vuelta en AuthorizationFilter:
    a. Si se lanzó KeyNotFoundException (token o usuario inválido), el filtro la captura y retorna un 401 Unauthorized.
    b. Compara el rol requerido por el atributo (ej. [AuthorizationFilter(RoleEnum.Administrator)]) con la lista user.Roles.
    c. Si el usuario no posee el rol, el filtro retorna un 403 Forbidden.
    d. Si el usuario posee el rol (o si el filtro no requería un rol específico), el método OnAuthorization termina sin retornar nada, permitiendo que la petición continúe hacia el AttractionsController.Get().

Filtros de Acción para Lógica Transversal (Autorización)

Entonces en lugar de repetir la lógica de validación de roles y sesión en cada método de cada controlador, se centralizó esta responsabilidad utilizando un Filtro de Acción personalizado.

- Implementación: La clase AuthorizationFilter intercepta las peticiones entrantes. Valida el token de sesión y comprueba si el rol del usuario autenticado corresponde con los roles permitidos para ese endpoint. Se aplica de forma declarativa sobre los controladores o acciones con el atributo [AuthorizationFilter(RoleEnum.Admin)].
- Impacto en Mantenibilidad y Seguridad: Este enfoque es un claro ejemplo del principio DRY (Don't Repeat Yourself). Reduce drásticamente el código duplicado y la posibilidad de errores humanos (olvidar una comprobación de seguridad). Si la lógica de autorización necesita cambiar en el futuro, se modifica en un único lugar, garantizando consistencia y robustez en la seguridad de toda la API.

Resumen de las Principales Mejoras al Diseño

Para cumplir con los nuevos requerimientos y mejorar la mantenibilidad de la solución, se realizaron cuatro refactorizaciones y mejoras principales sobre el diseño anterior.

1. **Segregación de Responsabilidades en AttractionAccessService (SRP)**

- En la versión anterior, el servicio AttractionAccessService era una clase "dios" que contenía lógica de:
  1. Búsqueda de tickets (por QR o por ID de visitante).
  2. Validación de disponibilidad (si la atracción tenía incidencias).
  3. Validación de edad del visitante.
  4. Registro del acceso.
- Se aplicó el **Principio de Responsabilidad Única (SRP)**. El servicio fue refactorizado para solo orquestar, y sus responsabilidades secundarias fueron extraídas a servicios dedicados y altamente cohesivos:
  - **TicketLookupService**: Ahora es el único responsable de la lógica de "cómo" encontrar un ticket (ya sea por QR o por Visitante+Fecha), actuando como el Experto (Grasp) que tiene la información (repositorios ITicketRepository y IVisitorRepository) para hacerlo.
  - **AttractionValidationService**: Ahora es el único responsable de determinar si una atracción es apta para el ingreso (validando edad y disponibilidad).
- Por otra parte, también se está aplicando DIP y ISP. El primero tal como ocurre en toda la aplicación en líneas generales; los módulos de alto nivel (Application) definen abstracciones (interfaces) que los módulos de bajo nivel (Infrastructure) implementan. El segundo, al definir interfaces específicas para cada cosa, y no una una "mega-interfaz".

Por último, se aplica el patrón **Facade**, que proporciona una interfaz unificada y simplificada a un conjunto de interfaces en un subsistema está siendo utilizado**.** Esto se ve claramente en el diagrama 1.

- **Justificación y Valor:** El AttractionAccessController (el cliente) no necesita conocer la complejidad de buscar un ticket, validar la edad, verificar incidencias, y registrar el acceso. AttractionAccessService actúa como una fachada que simplifica esta operación en un solo método (RegisterAccessByQrCode / RegisterAccessByVisitorId), coordinando internamente a ITicketLookupService, IAttractionValidationService, IScoringService, etc. Esto reduce el acoplamiento entre el controlador y los múltiples subsistemas de validación, y permite modificar el flujo de acceso (ej. añadir un nuevo paso de validación) sin cambiar la firma del controlador, adhiriendo a los principios de **Bajo Acoplamiento** y **Variaciones Protegidas** de GRASP.
  - **Valor Aportado:** AttractionAccessService ahora solo orquesta el flujo de acceso, delegando las validaciones y búsquedas. Esto lo hace más simple, más fácil de probar y más mantenible.

2. **Extensibilidad del Bloqueo de Atracciones (OCP)**
   - En la versión anterior, la única forma de que una atracción no estuviera disponible era mediante una Incidence creada manualmente por un operador.
   - **Requerimiento: Mantenimiento Preventivo:** Se necesitaba que un mantenimiento programado también bloqueará la atracción.
   - **Decisión de Diseño (OCP):** En lugar de modificar AttractionValidationService para que consulte Incidences y también PreventiveMaintenances (lo cual violaría el **Principio Abierto/Cerrado - OCP**), se decidió que el nuevo MaintenanceService genere una Incidence asociada al programar un mantenimiento.
   - **Valor Aportado:** AttractionValidationService (el código existente) permaneció cerrado a modificaciones. El sistema fue abierto a extensión, ya que la nueva funcionalidad de Mantenimiento se acopló a un concepto existente (Incidence) que el validador ya entendía.
3. **Separación de Lógica de Puntos (RewardService vs. ScoringService) (SRP)**
   - En la versión anterior, ScoringService era el centro de la gamificación y se encargaba de otorgar puntos.

- **Requerimiento: Canje de Recompensas:** Se necesitaba una forma de consumir o gastar puntos.
- **Decisión de Diseño (SRP):** En lugar de añadir la lógica de canjes (stock, membresía, costo) a ScoringService, se creó un servicio separado RewardService. Para mantener la consistencia del puntaje, ScoreLog se adaptó para aceptar puntos negativos (débitos) y ScoreLogRepository implementó TotalPointsByVisitor (un SUM()).
- **Valor Aportado:** Se mantiene una alta cohesión. ScoringService se enfoca en otorgar puntos (gamificación), mientras RewardService se enfoca en canjear puntos (lógica de recompensas). Ambos operan sobre el mismo log contable (ScoreLog).

Descripción del flujo en el frontend de VirtualPark-App

VirtualPark-App se desarrolla como una **Single Page Application (SPA)**, bajo el enfoque moderno de Angular basado en **Standalone Components** y configuración directa, prescindiendo por completo de la utilización de NgModules. Esta arquitectura implica que cada componente, servicio, ruta y proveedor se declara y utiliza de manera individual, lo cual favorece la simplicidad estructural, la mantenibilidad del código y una mayor eliminación de código innecesario en el empaquetado final.

Estructura, Componentes y Signals

La aplicación inicia su ejecución cargando el componente raíz (app.component.ts). Este componente referencia una plantilla principal y estilos, actuando como el punto de entrada para toda la interfaz. La configuración global se centraliza en app.config.ts, donde se integran los distintos proveedores: rutas, interceptores HTTP, manejo de errores globales y optimización del ciclo de detección de cambios de Angular.

Dentro de la carpeta src/app, cada funcionalidad está organizada en carpetas específicas, agrupando componentes que representan vistas, formularios o secciones completas, además de servicios dedicados y modelos de datos. La navegación se centraliza en el NavbarComponent, que actúa como marco persistente. Desde esta barra de navegación, el contenido desplegado al usuario es inyectado dinámicamente según la ruta activa, permitiendo al usuario navegar entre la visualización de atracciones, la gestión de usuarios, la compra de tickets y los módulos especializados de administración (como Mantenimiento o Reportes).

Para nuestro frontend, adoptamos Angular Signals sobre el patrón tradicional de subscribe, ya que creemos que representa un enfoque más moderno y nos facilitó drásticamente la gestión del estado. Encontramos que usar Signals simplifica la reactividad, eliminando la necesidad de gestionar manualmente suscripciones y desuscripciones, lo cual no solo vuelve el código más limpio y legible, sino que también previene las fugas de memoria. Al utilizar esta herramienta moderna, alineada con las últimas tendencias de la industria, logramos una arquitectura más robusta, eficiente y fácil de mantener, demostrando nuestra familiaridad con las prácticas actuales de Angular.

Enrutamiento y protección de vistas

El archivo app.routes.ts define de manera explícita la estructura jerárquica de rutas, donde cada ruta apunta directamente a componentes, prescindiendo también de módulos intermedios. Este archivo describe tanto las rutas públicas (login, registro) como las protegidas, que requieren autenticación y ciertos privilegios de rol.

La protección de rutas se implementa mediante **guardas** construidas como funciones (guestGuard, authChildGuard, roleGuard). Estos guardas controlan el acceso en función del estado de autenticación, la presencia y validez de un token, y los roles del usuario, todo lo cual se gestiona a nivel de servicios y almacenamiento local.

Servicios, lógica de negocio e interceptor

La lógica de negocio está encapsulada en **servicios**, declarados en la carpeta /services. El auth.service.ts se encarga de procesar el inicio y cierre de sesión, almacenar las credenciales (token y datos del usuario) en localStorage, y proporcionar información sobre el estado del usuario al resto de la aplicación.

El **interceptor** (auth.interceptor.ts) es registrado globalmente en la configuración, y se ocupa de inyectar automáticamente el token de sesión a las cabeceras de todas las peticiones HTTP efectuadas por la aplicación. Todo este vínculo se declara y configura sin NgModules, directamente en los archivos de configuración y en los propios componentes o servicios.

Credenciales, roles y localStorage

Al autenticarse correctamente, los datos del usuario y su token se almacenan en localStorage mediante el auth.service.ts. El sistema utiliza estos datos cada vez que se necesita verificar si el usuario tiene permisos para una vista o acción determinada (por ejemplo, accediendo a rutas protegidas por roles como "Administrator", "Operator" o "Visitor"). Los guardas consumen esta información y determinan dinámicamente la disponibilidad de rutas, menús y funcionalidades.

Al cerrar sesión, el sistema borra los datos de localStorage y redirige al usuario, asegurando consistencia y protección de datos sensibles.

Interfaces y tipado estricto

Todo el intercambio y procesamiento de datos se define a través de **interfaces** TypeScript ubicadas en /models, lo que garantiza seguridad de tipos y coherencia en el tránsito de información, tanto entre componentes y servicios como en la comunicación con el backend.

Identidad visual: integración de Tailwind CSS y Angular Material

La presentación y coherencia visuales se logran utilizando **Tailwind CSS** para utilidades rápidas (declarándose en styles.css) y **Angular Material** para componentes robustos y *theming* (custom-theme.scss). La configuración de Material Design se efectúa con los nuevos esquemas de CSS variables y mixins, adaptándose eficientemente a la arquitectura moderna de Angular y permitiendo una personalización sencilla, acorde a los estándares actuales de diseño.

Carga dinámica de estrategias de puntaje

El sistema VirtualPark implementa un mecanismo de carga dinámica de estrategias de puntaje mediante Reflection que permite agregar nuevas estrategias en tiempo de ejecución sin necesidad de recompilar la aplicación. Las estrategias se cargan desde archivos DLL ubicados en la carpeta /Plugins.

Interfaz para Desarrolladores Externos

Todo desarrollador que desee crear una nueva estrategia de puntaje debe implementar la interfaz *IScoringStrategy*, ubicada en el namespace *VirtualPark.Application.Scoring*.

Ubicación: VirtualPark.Application/Scoring/IScoringStrategy.cs

```
public interface IScoringStrategy
{
    string Name { get; }

    int CalculatePoints(
        Attraction attraction,
        Visitor visitor,
        IEnumerable<AttractionAccess> todayAccesses,
        Domain.SpecialEvent? activeEvent);
}
```

Descripción de la Interfaz

Propiedad Name

- Tipo: string
- Descripción: Identificador único de la estrategia
- Requisitos:
    - Debe ser un nombre descriptivo y único
    - No puede coincidir con estrategias existentes (built-in o de otros plugins)
    - Se recomienda usar PascalCase (ej: PuntuacionPorHora, ScoreByVisitorAge)

Método CalculatePoints

- Retorno: int - Puntos a otorgar al visitante
- Parámetros:
    - attraction: La atracción que está siendo visitada
    - visitor: El visitante que accede a la atracción
    - todayAccesses: Colección de todos los accesos del visitante en el día actual
    - activeEvent: Evento especial activo del día (puede ser null)

Guía para Crear una Nueva Estrategia

Paso 1: Crear un Proyecto de Librería de Clases

Crear un nuevo proyecto .NET 8.0:

```
dotnet new classlib -n VirtualPark.Plugins.MiEstrategia
cd VirtualPark.Plugins.MiEstrategia
```

Paso 2: Agregar Referencias Requeridas

Editar el archivo .csproj para incluir las dependencias necesarias:

Paso 3: Implementar IScoringStrategy

Crear una clase que implemente la interfaz:

```
using VirtualPark.Domain;
using VirtualPark.Application.Scoring;

namespace VirtualPark.Plugins.MiEstrategia;

public class MiEstrategiaPersonalizada : IScoringStrategy
{
    public string Name => "MiEstrategiaPersonalizada";

    public int CalculatePoints(
        Attraction attraction,
        Visitor visitor,
        IEnumerable<AttractionAccess> todayAccesses,
        Domain.SpecialEvent? activeEvent)
    {
        int basePoints = 10;

        if (attraction.Type == AttractionType.Thrill)
        {
            basePoints += 20;
        }

        return basePoints;
    }
}
```

Requisitos importantes:

- La clase debe ser pública (public)
- La clase debe tener un constructor sin parámetros (constructor por defecto)
- La clase no debe ser abstracta

Paso 4: Compilar el Plugin

```
dotnet build -c Release
```

El archivo DLL se generará en: bin/Release/net8.0/NombreDelProyecto.dll

Paso 5: Desplegar el Plugin

Copiar el archivo DLL generado a la carpeta /Plugins del sistema VirtualPark:
La misma se puede hacer upload desde la UI también si así se desea.

Paso 6: Activar la Estrategia

Una vez cargado el plugin, la estrategia estará disponible en el sistema. Para activarla:

1. Acceder al panel de administración de estrategias en la interfaz web
2. La nueva estrategia aparecerá automáticamente en la lista de estrategias disponibles
3. Seleccionar la estrategia para activarla

Mecanismo de Descubrimiento por Reflection

Clase ScoringStrategyLoader

Ubicación: VirtualPark.Application/Scoring/ScoringStrategyLoader.cs

Esta clase es responsable de:

1. Cargar las estrategias built-in (integradas en el código)
2. Explorar la carpeta /Plugins buscando archivos .dll
3. Usar Reflection para identificar tipos que implementan IScoringStrategy
4. Instanciar y registrar dinámicamente las estrategias encontradas

Proceso de Carga

1. Al iniciar la aplicación, se configura la ruta de plugins (ServiceCollectionExtensions.cs).
2. Cuando se solicita la lista de estrategias disponibles (GetStrategies()):
    - Se cargan primero las estrategias integradas
    - Se escanea la carpeta /Plugins buscando archivos .dll
    - Para cada DLL encontrado, se usa Assembly.LoadFrom() para cargar el ensamblado
    - Se filtran tipos que cumplan los requisitos de IScoringStrategy
    - Se instancian mediante Activator.CreateInstance()
    - Se registran en el diccionario de estrategias disponibles
3. Las estrategias quedan disponibles inmediatamente para su activación

Integración con la Interfaz Gráfica

Diseño Preparado para Extensibilidad

La interfaz gráfica está diseñada para no requerir modificaciones al agregar nuevas estrategias. La misma, no recibe los distintos nombres de estrategias hardcodeadas, sino

que los obtiene haciendo un reques GETt a la API para obtener las estrategias disponibles, y así hacer display de las estrategias disponibles para su elección,

Funcionalidad de Carga de Plugins desde UI

La interfaz incluye un componente para subir archivos DLL directamente.
Proceso:

1. El usuario selecciona un archivo DLL desde su sistema
2. El archivo se envía mediante POST /api/Scoring/strategies/plugins
3. El servidor guarda el DLL en la carpeta /Plugins
4. La estrategia queda disponible inmediatamente
5. Al refrescar la lista, la nueva estrategia aparece automáticamente

API REST para Gestión de Estrategias

Controller: ScoringController (VirtualPark.WebApi/Controllers/ScoringController.cs)

Endpoints Disponibles

Obtener Estrategias Disponibles

```
GET /api/Scoring/strategies
Authorization: Administrator
```

Activar una Estrategia

```
PUT /api/Scoring/strategies/active
Authorization: Administrator
Content-Type: application/json
```

Subir un Plugin

```
POST /api/Scoring/strategies/plugins
Authorization: Administrator
Content-Type: multipart/form-data

plugin: [archivo .dll]
```

Ventajas del Diseño Implementado

1. Extensibilidad: Terceros pueden agregar estrategias sin acceso al código fuente
2. Sin recompilación: Las estrategias se cargan en tiempo de ejecución
3. Desacoplamiento: Las estrategias externas no dependen de la implementación interna del sistema
4. UI preparada: La interfaz se adapta automáticamente a nuevas estrategias
5. Validación: El sistema valida automáticamente la compatibilidad de los plugins
6. Persistencia: La estrategia activa se persiste en base de datos

Justificaciones Propias

1. Se determina que los usuarios visitantes, al realizar modificaciones, están habilitados para cambiar todos sus atributos, con excepción de la fecha de nacimiento y su nivel de membresía.
2. Se implementa una política de una única sesión activa por usuario: al iniciar sesión, se procederá a la eliminación de cualquier sesión previa del mismo usuario.
3. El proceso de registro genera exclusivamente objetos de tipo Visitor, y no Users "genéricos": el método AuthService.Register() crea un Visitor con el rol Visitor. **Premisa:** el registro público está destinado solo a visitantes.
4. La actualización de perfil admite una contraseña opcional: si se proporciona, esta será hasheada y persistida; en caso contrario, se mantendrá la contraseña existente. El *hashing* se realiza mediante el algoritmo PBKDF2-HMACSHA256.
5. Las capacidades asignadas a los eventos son **"**absolutas" y globales al evento en sí, no estando segmentadas por fecha/hora específica dentro del evento ni por franja horaria.
6. Los roles administran el acceso a los *endpoints* (como *Reports* y *Users*) a través de un filtro dedicado; los *tokens* no poseen caducidad.
7. Durante la actualización del perfil, tanto el correo electrónico como el nombre/apellido pueden ser modificados sin requerir una reconfirmación; la contraseña es opcional.
8. Se establece que el código QR es un GUID como identificador único, en consonancia con lo sugerido en el foro.
9. Se define un único *endpoint* para gestionar ambos tipos de *tickets* (POST /api/tickets), con la inclusión condicional de los parámetros Type y SpecialEventId.
10. Los DTOs y los *mappings* se alojan en un proyecto separado; la función de mapeo se invoca en el *controller*, y al *service* se le transfiere directamente el objeto de dominio.
11. Se implementa un filtro global que mapea las excepciones de la siguiente manera:
    - ArgumentException/InvalidOperationException a un código 400
    - KeyNotFoundException a un 404
    - DuplicateEmailException a un 409
    - InvalidCredentialsException a un 401.
    - Las excepciones DbUpdateException se mapean a un 400, incluyendo detalles. Los mensajes de error proporcionados son deliberadamente poco explícitos para evitar la divulgación de información sensible.

Uso de IA

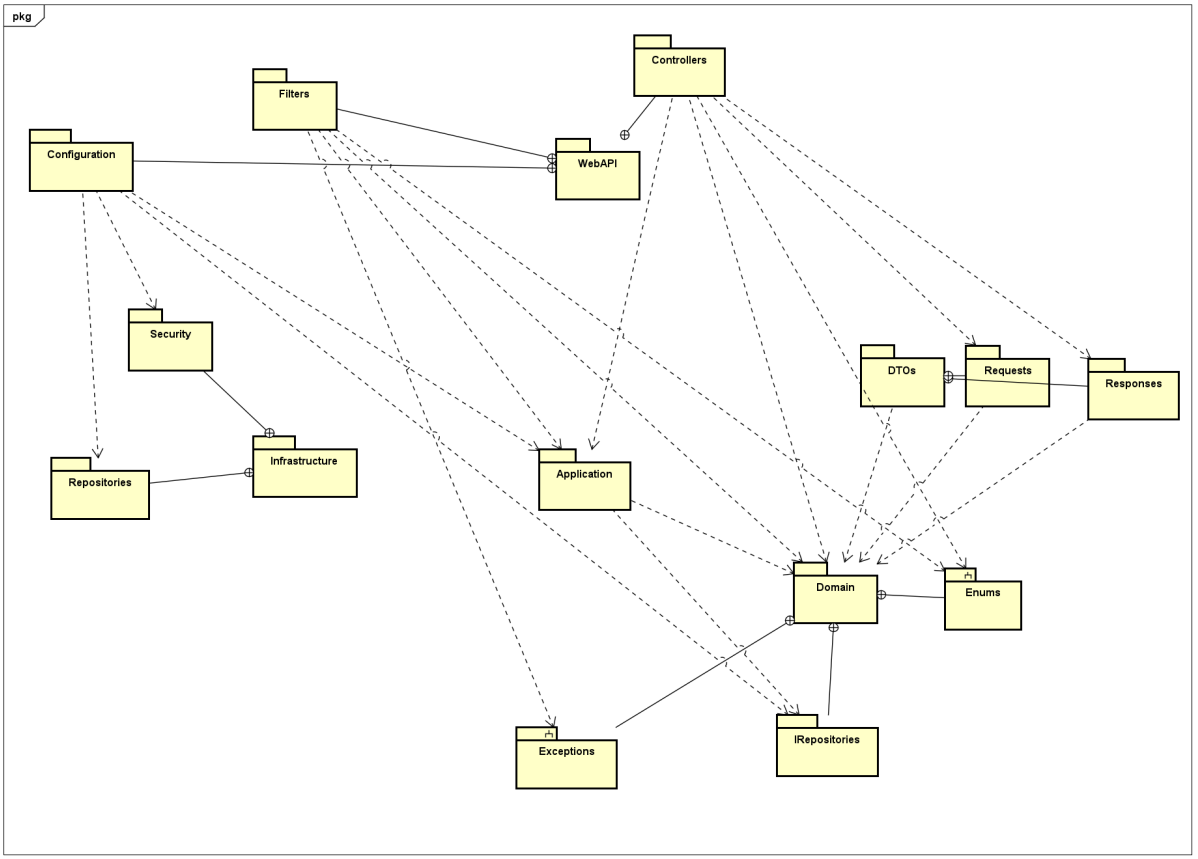
Para el desarrollo del obligatorio se utilizaron diferentes modelos y se fue alternando para obtener los mejores resultados.
Modelos utilizados: claude-4.5, GPT-5 mini, claude Sonnet 4, Gemini 2.5 Pro, GPT-5-Codex, GPT-4.1, GPT-5 Chat, GPT-5, Composer-1.

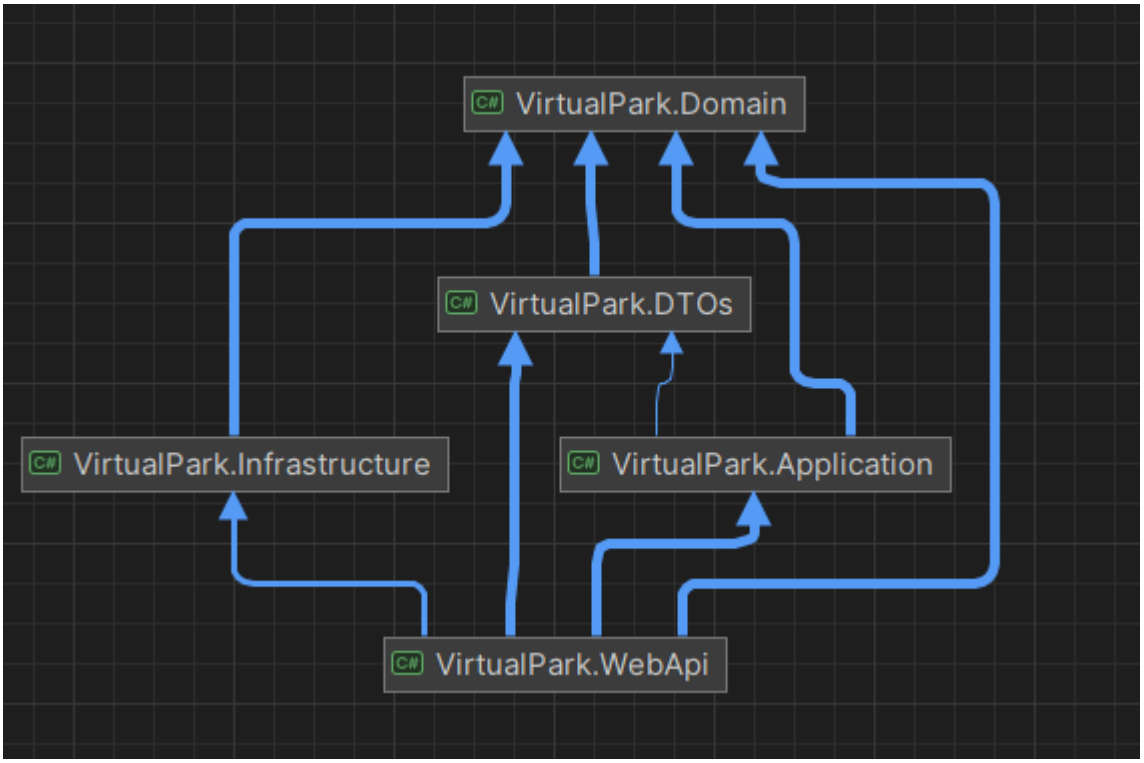
Especificación API

Se realizó una siguiente tabla demostrando la especificación de la API, incluyendo solamente los endpoints que tuvieron cambios en esta versión.
Se puede acceder a la tabla en el anexo: 310896-313578-303636 Obligatorio 2

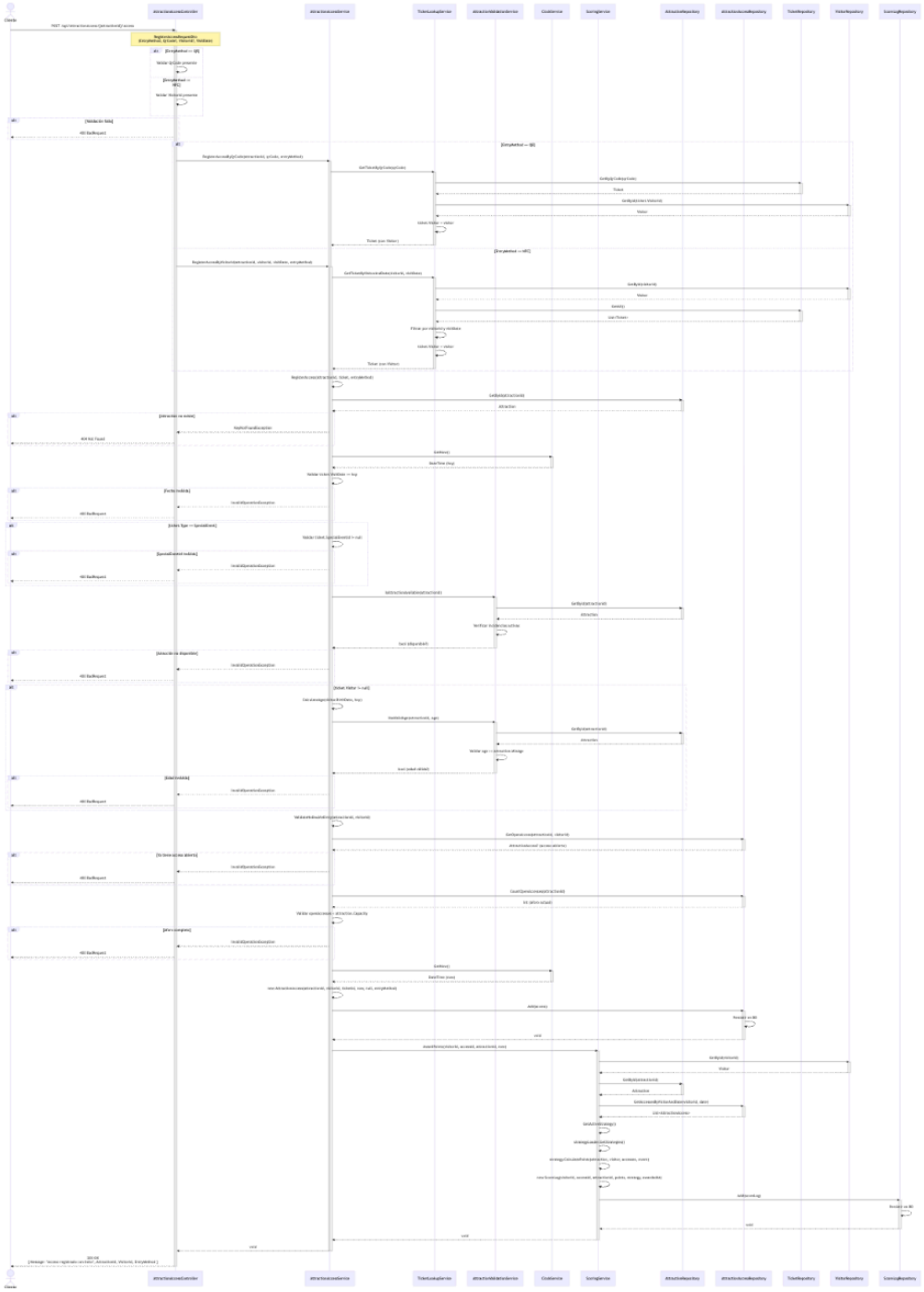
ANEXO

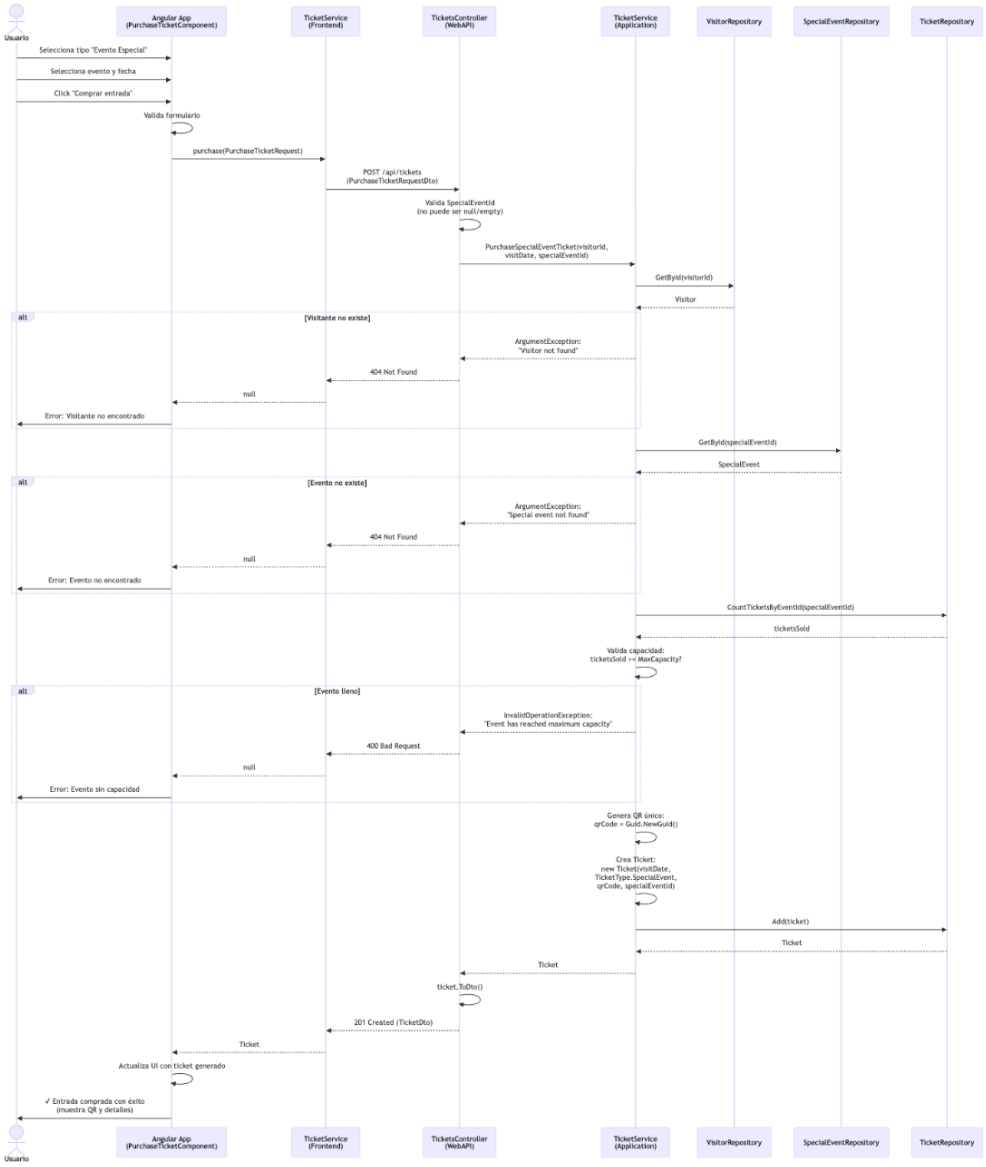
Diagrama de paquetes:

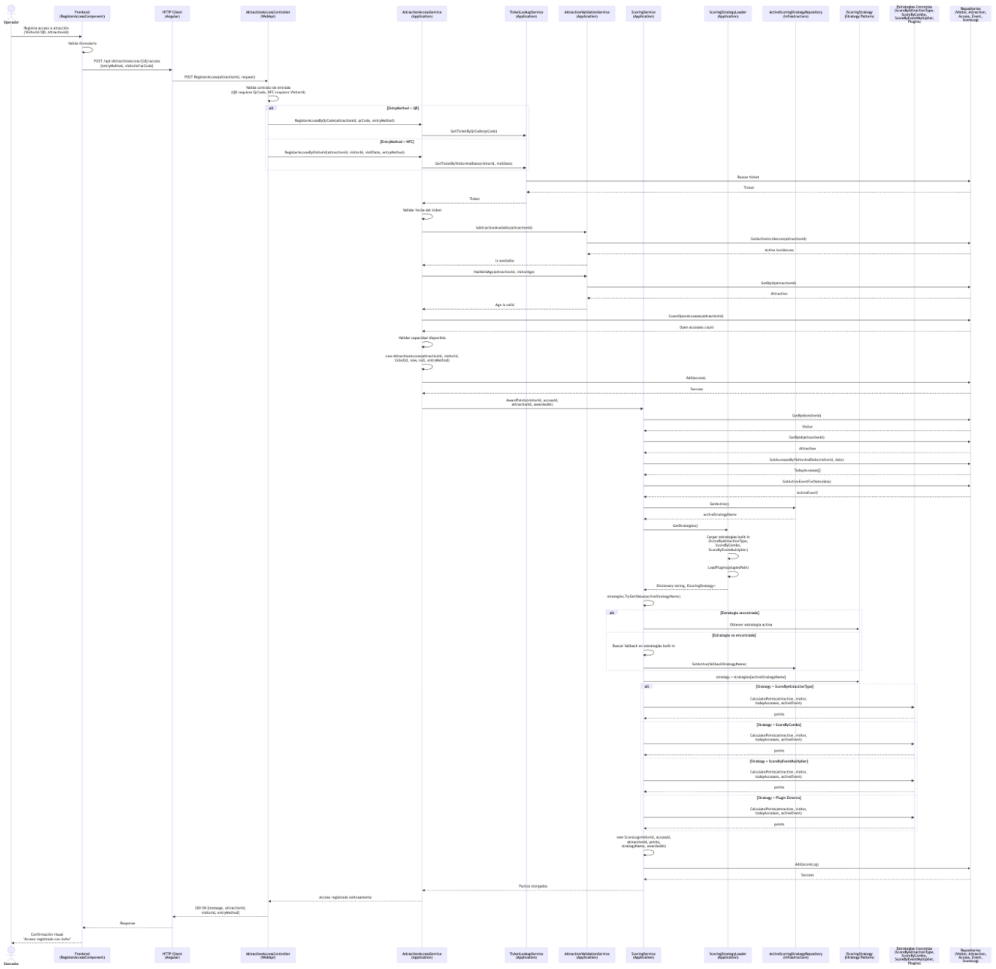



VirtualPark.Domain
VirtualPark.DTOs
VirtualPark.Infrastructure
VirtualPark.Application
VirtualPark.WebApi

Diagramas de secuencia




Usuario
Angular App (PurchaseTicketComponent)
TicketService (Frontend)
TicketsController (WebAPI)
TicketService (Application)
VisitorRepository
SpecialEventRepository
TicketRepository
Selecciona tipo "Evento Especial"
Selecciona evento y fecha
Click "Comprar entrada"
Valida formulario
purchase(PurchaseTicketRequest)
POST /api/tickets (PurchaseTicketRequestDto)
Valida SpecialEventId (no puede ser null/empty)
PurchaseSpecialEventTicket(visitorId, visitDate, specialEventId)
GetById(visitorId)
Visitor
alt
[Visitante no existe]
ArgumentException: "Visitor not found"
404 Not Found
null
Error: Visitante no encontrado
GetById(specialEventId)
SpecialEvent
alt
[Evento no existe]
ArgumentException: "Special event not found"
404 Not Found
null
Error: Evento no encontrado
CountTicketsByEventId(specialEventId)
ticketsSold
Valida capacidad: ticketsSold >= MaxCapacity?
alt
[Evento lleno]
InvalidOperationException: "Event has reached maximum capacity"
400 Bad Request
null
Error: Evento sin capacidad
Genera QR único: qrCode = Guid.NewGuid()
Crea Ticket: new Ticket(visitDate, TicketType.SpecialEvent, qrCode, specialEventId)
Add(ticket)
Ticket
Ticket
ticket.ToDto()
201 Created (TicketDto)
Ticket
Actualiza UI con ticket generado
✓ Entrada comprada con éxito (muestra QR y detalles)



En la siguiente carpeta se tienen los diagramas de secuencia en formato .svg para facilitar su visualización.
https://drive.google.com/drive/folders/1kg4MtE54g7UJgIPlhqtHc6CXFiSTYplF?usp=sharing

Diagrama de componentes

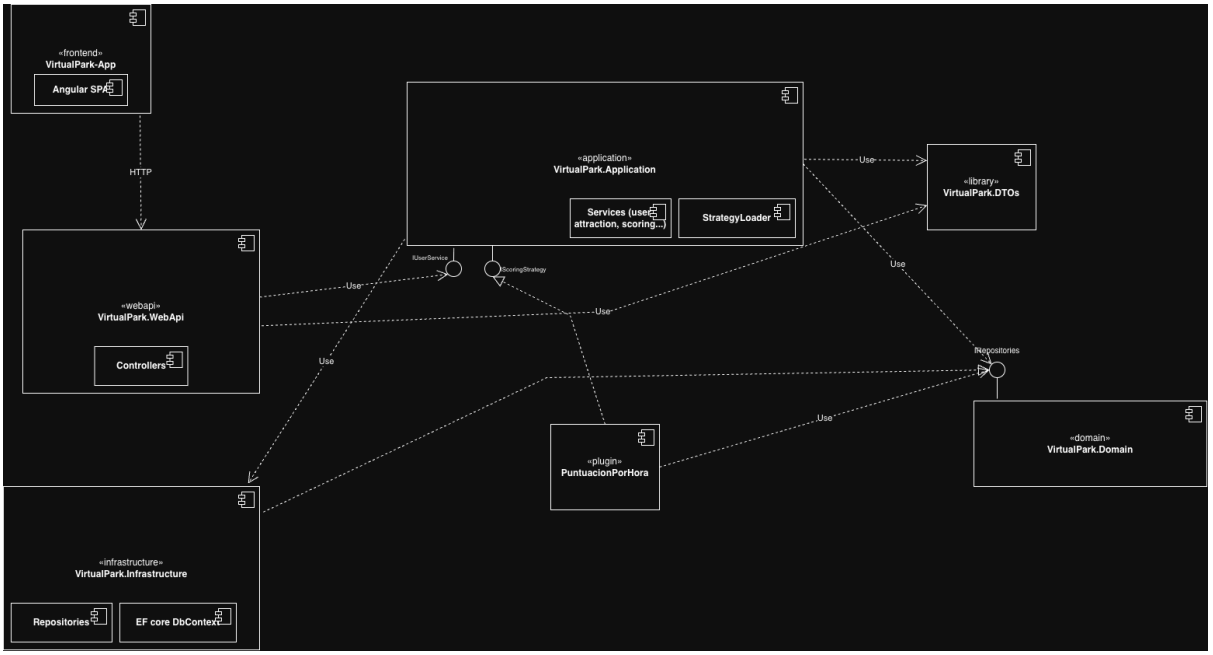
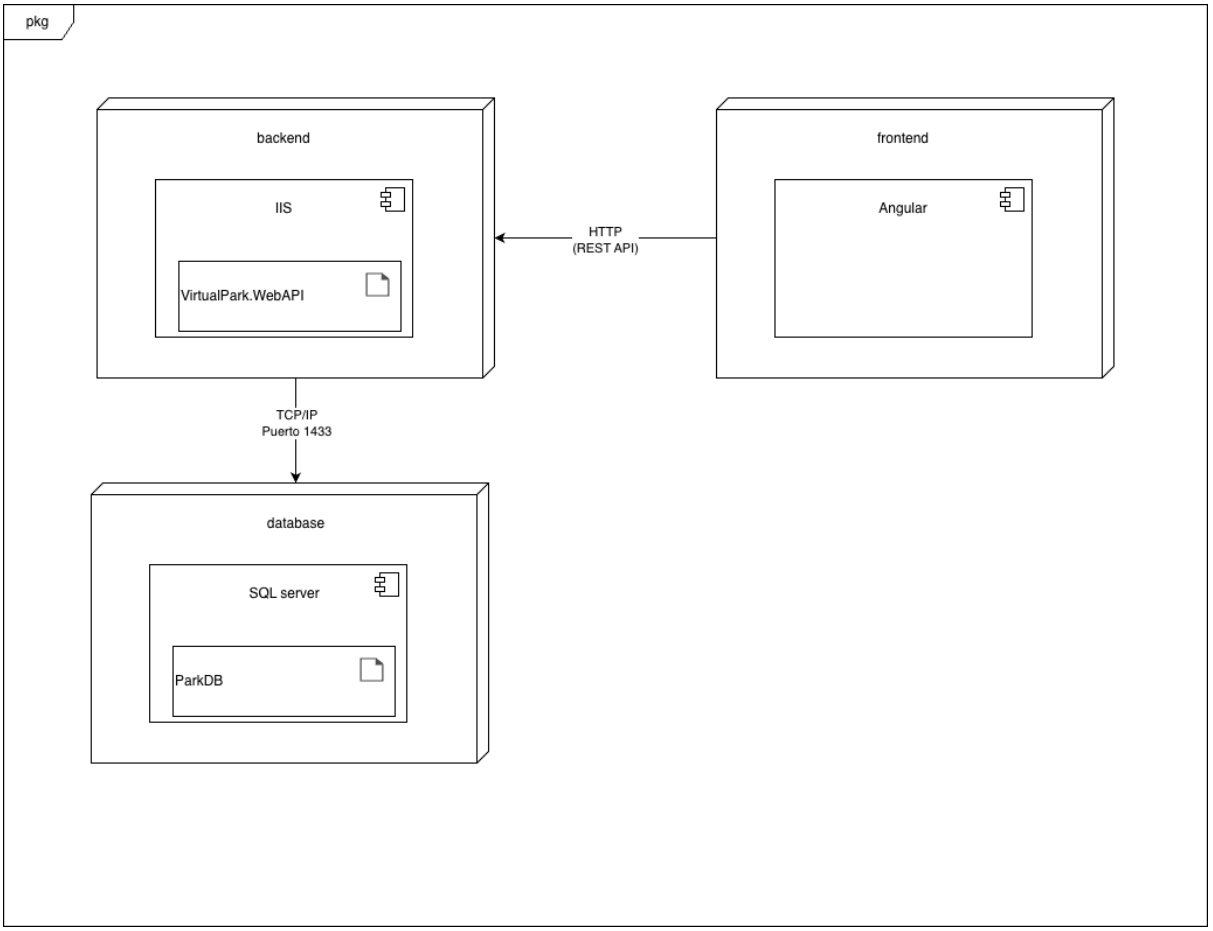


Diagrama de despliegue



Url para ver la imagen con mayor claridad:
https://drive.google.com/file/d/1JgIhxnc2e3tgYJ7BERPVjaxSh4nVTRp_/view?usp=sharing.

Especificación API

Path	Método	Headers	Params	Rol	Request Body	Response Body	Response Codes	Descripción
`/api/Scoring/strategies/plugins`	`POST`	`Authorization: Bearer {token}Content-Type: multipart/form-data`	N/A	Administrador	Form-data con campo `plugin` (archivo DLL)	**200 OK**: `{ "Message": "Plugin '{fileName}' cargado correctamente." }`**400 Bad Request**: `{ "Error": "Se requiere un archivo DLL." }` o `{ "Error": "{error message}" }`**500 Internal Server Error**: `{ "Error": "No se pudo guardar el plugin..." }`	`200` - Plugin cargado`400` - Solicitud inválida`401` - No autorizado `500` - Error del servidor	Permite a los administradores cargar un archivo DLL que contiene una nueva estrategia de puntuación personalizada. El plugin es almacenado en el directorio de Plugins.
`/api/RewardRedemption/visitor/{visitorId}`	`GET`	`Authorization: Bearer {token}`	`visitorId` (Guid, path)	Visitor	N/A	**200 OK**: `[{ "Id": "guid", "RewardId": "guid", "VisitorId": "guid", "PointsSpent": int, "Date": "datetime" }]`	`200` - Canjes recuperados`401` - No autorizado	Obtiene el historial completo de canjes de recompensas realizados por un visitante específico, incluyendo la recompensa canjeada, puntos gastados y fecha.
`/api/Reward`	`POST`	`Authorization: Bearer`	N/A	Administrador	`{ "Name": "string"`	**201 Created**: Objeto	`201` - Recompensa	Crea una nueva recompens

		{token}Content-Type: application/json			, "Description": "string" , "CostInPoints": int, "AvailableQuantity": int, "RequiredLevel": "Bronze|Silver|Gold|Platinum|null" }	Reward con Id generado	creada400 - Datos inválidos401 - No autorizado	a en el sistema que los visitantes pueden canjear con sus puntos acumulados. Permite especificar un nivel de membresía requerido opcional.
/api/Reward	GET	N/A	N/A	Ninguno (público)	N/A	**200 OK**: [{ "Id": "guid", "Name": "string" , "Description": "string" , "CostInPoints": int, "AvailableQuantity": int, "RequiredLevel": "MembershipLevel|null" }]	200 - Recompensas recuperadas	Obtiene la lista completa de todas las recompensas disponibles en el parque, incluyendo su costo en puntos, cantidad disponible y nivel requerido.
/api/Reward/{rewardId}/redeem	POST	Authorization: Bearer {token}	rewardId (Guid, path)	Visitor	N/A	**200 OK**: { "Message": "Recompensa canjeada con éxito", "redemption": { RewardRedemption } }**401 Unauthorized**: Sin token o sesión inválida**400 Bad Request**:	200 - Recompensa canjeada400 - Error en el canje401 - No autorizado	Permite a un visitante canjear una recompensa utilizando sus puntos acumulados. El visitante se obtiene del token. Valida puntos suficientes y disponibilidad.

						Puntos insuficientes, etc.		
/api/Maintenance	POST	Authorization: Bearer {token}Content-Type: application/json	N/A	Administrator	{ "AttractionId": "guid", "ScheduledDate": "datetime", "StartTime": "timespan", "EstimatedDuration": "timespan", "Description": "string (3-1000 chars)" }	**201 Created**: { "Id": "guid", "AttractionId": "guid", "ScheduledDate": "datetime", "StartTime": "timespan", "EstimatedDuration": "timespan", "Description": "string" , "AssociatedIncidenceId": "guid|null" }	201 - Mantenimiento programado400 - Datos inválidos401 - No autorizado	Programa un mantenimiento preventivo para una atracción específica. Permite definir la fecha, hora de inicio, duración estimada y descripción del trabajo.
/api/Maintenance/{id}	GET	Authorization: Bearer {token}	id (Guid, path)	Cualquier autenticado	N/A	**200 OK**: MaintenanceDto**404 Not Found**: Mantenimiento no encontrado	200 - Mantenimiento encontrado404 - No existe401 - No autorizado	Obtiene los detalles completos de un mantenimiento específico mediante su ID único.
/api/Maintenance/attraction/{attractionId}	GET	Authorization: Bearer {token}	attractionId (Guid, path)	Cualquier autenticado	N/A	**200 OK**: Array de MaintenanceDto	200 - Mantenimientos recuperados401 - No autorizado	Obtiene todos los mantenimientos programados (pasados y futuros) para una atracción específica.
/api/Maintenance/{id}	DELETE	Authorization: Bearer {token}	id (Guid, path)	Administrator	N/A	**204 No Content**: Mantenimiento cancelado **404 Not Found**: No encontrado	204 - Cancelado exitosamente404 - No existe401 - No autorizado	Cancela un mantenimiento programado. El mantenimiento es eliminado

								del sistema.
/api/Visitors/{visitorId}/score-history	GET	Authorization: Bearer {token}	visitorId (Guid, path)	Visitor	N/A	**200 OK**: [{ "Id": "guid", "Date": "datetime", "PointsAwarded": int, "Origin": "string", "StrategyUsed": "string", "AttractionName": "string|null", "Description": "string" }]**404 Not Found**: { "message": "Visitante no encontrado" }	200 - Historial recuperado404 - Visitante no encontrado401 - No autorizado	Obtiene el historial completo de puntos ganados por un visitante, incluyendo fecha, puntos, origen (atracción), estrategia utilizada y descripción.
/api/AttractionAccess/{attractionId}/exit	PUT	Authorization: Bearer {token}	attractionId (Guid, path)visitorId (Guid, query)	Operator	N/A	**200 OK**: { "Message": "Salida registrada con éxito", "AttractionId": "guid", "VisitorId": "guid", "ExitTime": "datetime" }	200 - Salida registrada400 - Error en solicitud401 - No autorizado	Registra la salida de un visitante de una atracción. Actualiza el registro de acceso existente agregando la hora de salida del sistema. **Endpoint refactorizado para ser RESTful.**
/api/AttractionAccess/{attractionId}/incidents	PUT	Authorization: Bearer {token}	attractionId (Guid, path)incidentId (Guid, query)action (string,	Operator	N/A	**200 OK**: { "Message": "Incidencia cerrada con éxito" }	200 - Acción ejecutada400 - Acción inválida401 - No autorizado	Actualiza el estado de una incidencia existente. Permite cerrar (action=close) o

			query): "close" o "reopen"			o { "Message": "Incidencia reabierta con éxito" }**400 Bad Request**: { "Message": "Acción no válida..." }		reabrir (action=reopen) una incidencia. **Endpoint refactorizado para ser RESTful.**
/api/AttractionAccess/{attractionId}/incidents	GET	Authorization: Bearer {token}	attractionId (Guid, path)incidentId (Guid, query, opcional)	Operator	N/A	**200 OK**: IncidenceDto individual (si incidentId presente) o Array de IncidenceDtoIncidenceDto: { "Id": "guid", "Title": "string", "Description": "string", "Status": boolean, "Date": "datetime", "AttractionId": "guid", "AttractionName": "string|null" }**404 Not Found**: { "Message": "Incidencia no encontrada" }	200 - Incidencias recuperadas400 - Error404 - Incidencia no encontrada401 - No autorizado	Obtiene incidencias de una atracción. Si se proporciona incidentId, retorna una específica. Si no, retorna todas las incidencias. **Endpoint refactorizado para ser RESTful.**
/api/SpecialEvent/{event	DELETE	Authorization: Bearer	eventId (Guid, path)att	Administrator	N/A	**200 OK**: SpecialEventDetailDto	200 - Atracción removida4	Remueve una atracción de un

`Id}/attractions`		`{token}`	`raction Id` (Guid, query)			actualizado**404 Not Found**: Evento/atracción no encontrados**400 Bad Request**: Error en operación	`00` - Error`404` - No encontrado`401` - No autorizado	evento especial. La atracción deja de estar asociada al evento pero permanece en el sistema. **Endpoint refactorizado para ser RESTful.**

Se disponibiliza un archivo de Excel para una mejor visualizacion de la misma: https://docs.google.com/spreadsheets/d/1ukoYXqm5PsVlHfE-lg261bfGBI6mxvGF5PtZCQjY0OY/edit?usp=sharing.

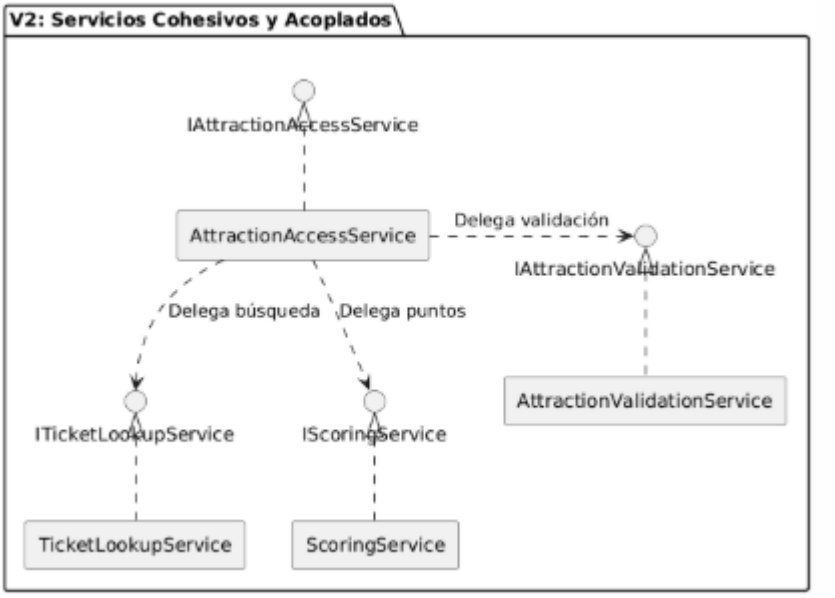
Resultados de métricas por assembly

Proyecto	A (Abstractness)	I (Instability)	D (Distance)	H (Relational Cohesion)
VirtualPark.Domain	0.43	0.25	0.23	0.86
VirtualPark.DTOs	0	0.77	0.16	0.71
VirtualPark.Application	0.43	0.86	0.20	0.88
VirtualPark.Infrastructure	0	0.99	0.01	1.05
VirtualPark.WebApi	0	1	0	1.27

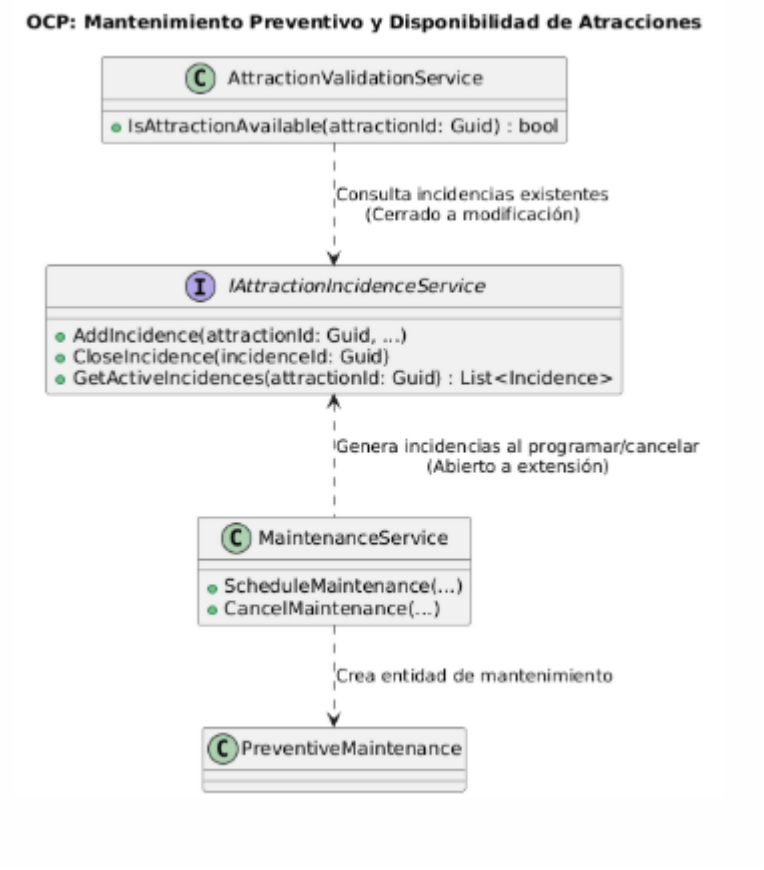
Diagramas de patrones

1.

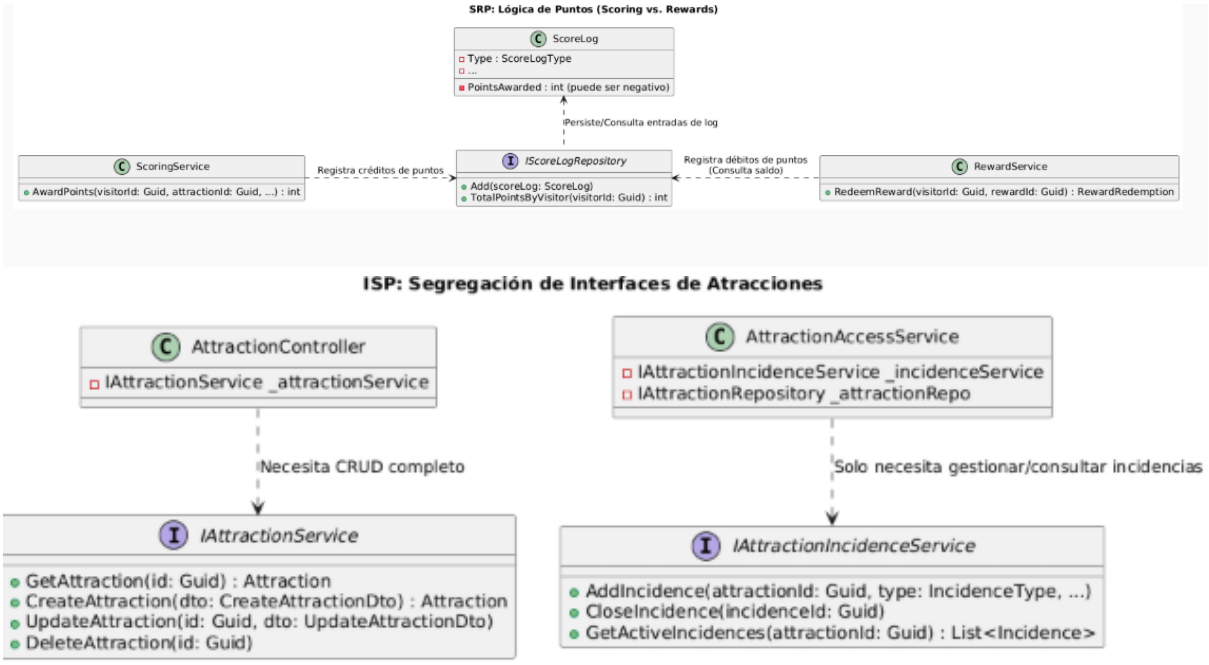
2.



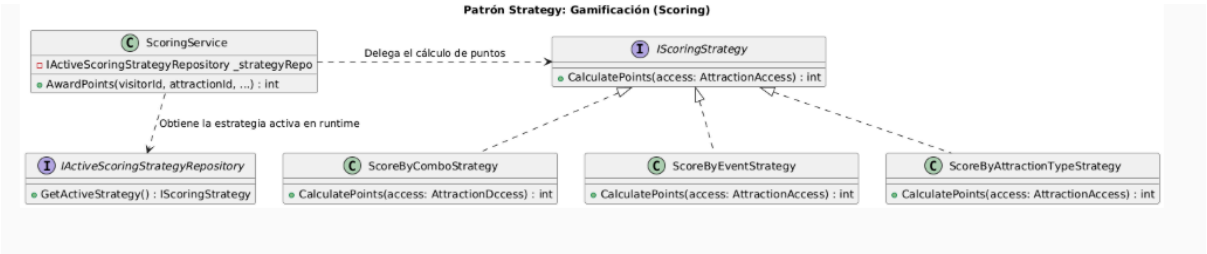
3.


OCP: Mantenimiento Preventivo y Disponibilidad de Atracciones
AttractionValidationService
IsAttractionAvailable(attractionId: Guid) : bool
Consulta incidencias existentes
(Cerrado a modificación)
IAttractionIncidenceService
AddIncidence(attractionId: Guid, ...)
CloseIncidence(incidenceId: Guid)
GetActiveIncidences(attractionId: Guid) : List<Incidence>
Genera incidencias al programar/cancelar
(Abierto a extensión)
MaintenanceService
ScheduleMaintenance(...)
CancelMaintenance(...)
Crea entidad de mantenimiento
PreventiveMaintenance

4.

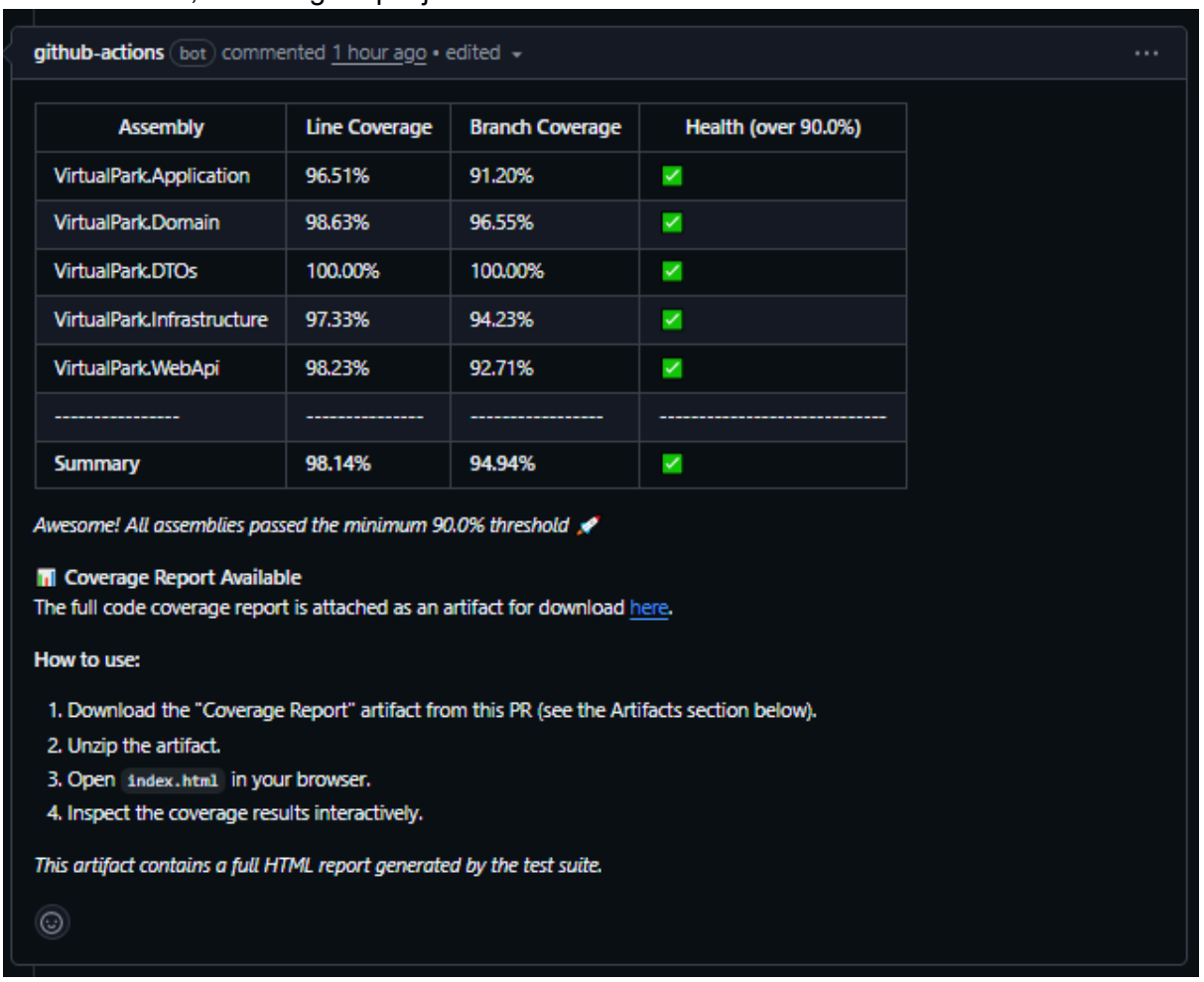

SRP: Lógica de Puntos (Scoring vs. Rewards)
ScoreLog
Type : ScoreLogType
...
PointsAwarded : int (puede ser negativo)
Persiste/Consulta entradas de log
ScoringService
AwardPoints(visitorId: Guid, attractionId: Guid, ...) : int
Registra créditos de puntos
IScoreLogRepository
Add(scoreLog: ScoreLog)
TotalPointsByVisitor(visitorId: Guid) : int
Registra débitos de puntos
(Consulta saldo)
RewardService
RedeemReward(visitorId: Guid, rewardId: Guid) : RewardRedemption
ISP: Segregación de Interfaces de Atracciones
AttractionController
IAttractionService _attractionService
Necesita CRUD completo
IAttractionService
GetAttraction(id: Guid) : Attraction
CreateAttraction(dto: CreateAttractionDto) : Attraction
UpdateAttraction(id: Guid, dto: UpdateAttractionDto)
DeleteAttraction(id: Guid)
AttractionAccessService
IAttractionIncidenceService _incidenceService
IAttractionRepository _attractionRepo
Solo necesita gestionar/consultar incidencias
IAttractionIncidenceService
AddIncidence(attractionId: Guid, type: IncidenceType, ...)
CloseIncidence(incidenceId: Guid)
GetActiveIncidences(attractionId: Guid) : List<Incidence>

5.



Informe de cobertura y análisis interpretativo de los resultados

Como se ve en la captura, cumplimos con los requisitos de tener más de 90% tanto de line como de branch coverage.
En el caso de el proyecto WebApi, la falta de cobertura se debe a falta de branch tests en el middleware de manejo de excepciones, las cuales para resolverlos deberíamos usar reflection u alguna otra estrategia para setear los tests para los casos que necesitáramos.
Por otra parte, en el paquete de Infrastructure, la falta de cobertura se debe a la clase HashPassword. En esta, se utiliza el “hashing con sal”, y se cubren la mayoría de líneas. La cobertura faltante se escapa a nuestros conocimientos y creemos que es intrínseca de la verificación de la password.
Por último en Application la falta de cobertura se debe a casos en los cuales deberíamos usar reflection para los tests de branch coverage, como en users o el scoring.
A continuación, una imagen que justifica la cobertura:



github-actions bot commented 1 hour ago • edited

Assembly	Line Coverage	Branch Coverage	Health (over 90.0%)
VirtualPark.Application	96.51%	91.20%	✅
VirtualPark.Domain	98.63%	96.55%	✅
VirtualPark.DTOs	100.00%	100.00%	✅
VirtualPark.Infrastructure	97.33%	94.23%	✅
VirtualPark.WebApi	98.23%	92.71%	✅
---	---	---	---
Summary	**98.14%**	**94.94%**	✅

Awesome! All assemblies passed the minimum 90.0% threshold 🚀

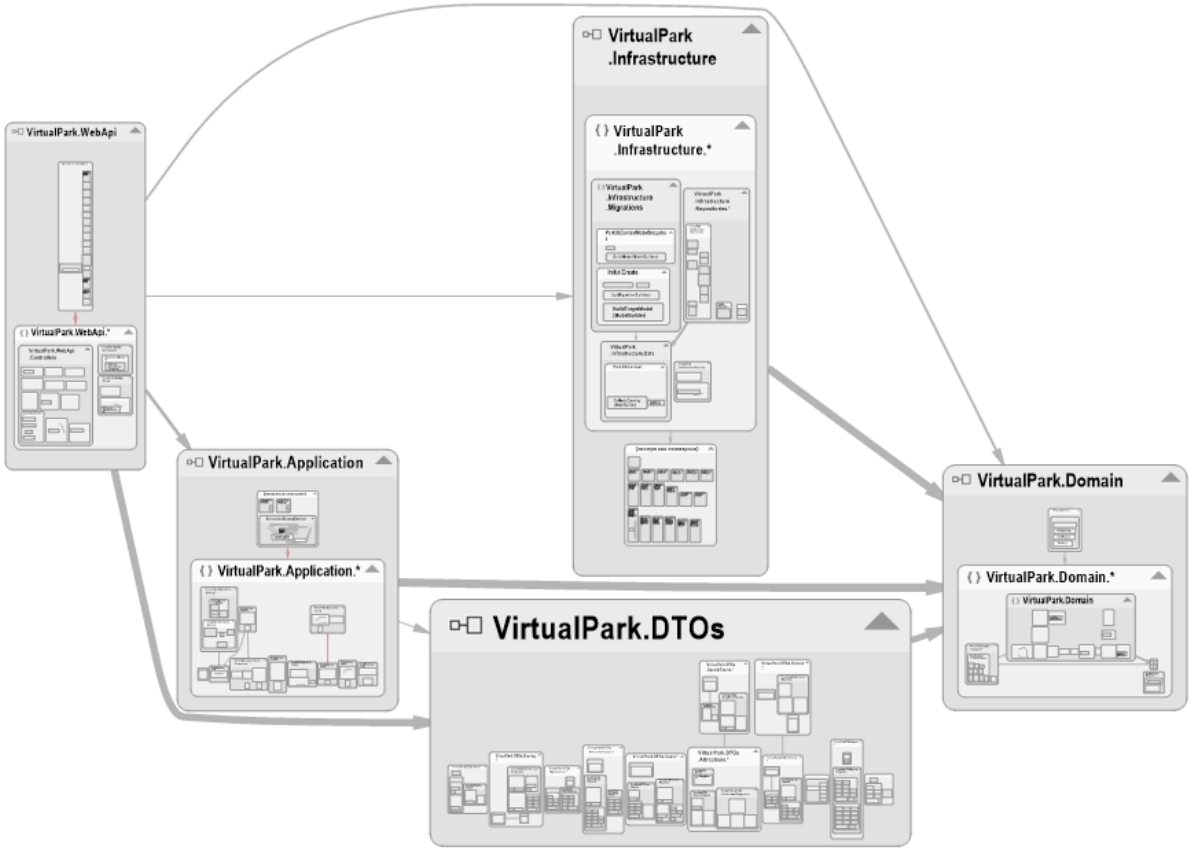
📊 **Coverage Report Available**
The full code coverage report is attached as an artifact for download here.

How to use:

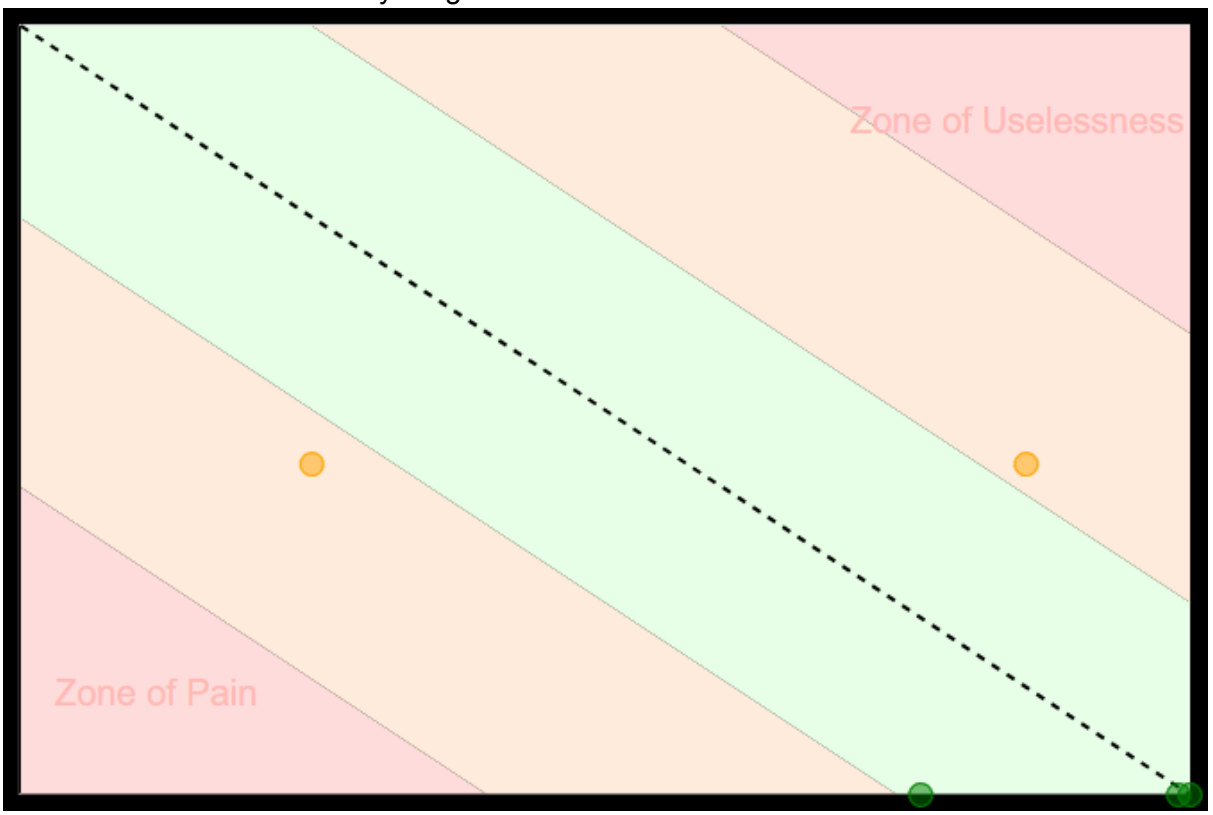
1. Download the "Coverage Report" artifact from this PR (see the Artifacts section below).
2. Unzip the artifact.
3. Open `index.html` in your browser.
4. Inspect the coverage results interactively.

This artifact contains a full HTML report generated by the test suite.

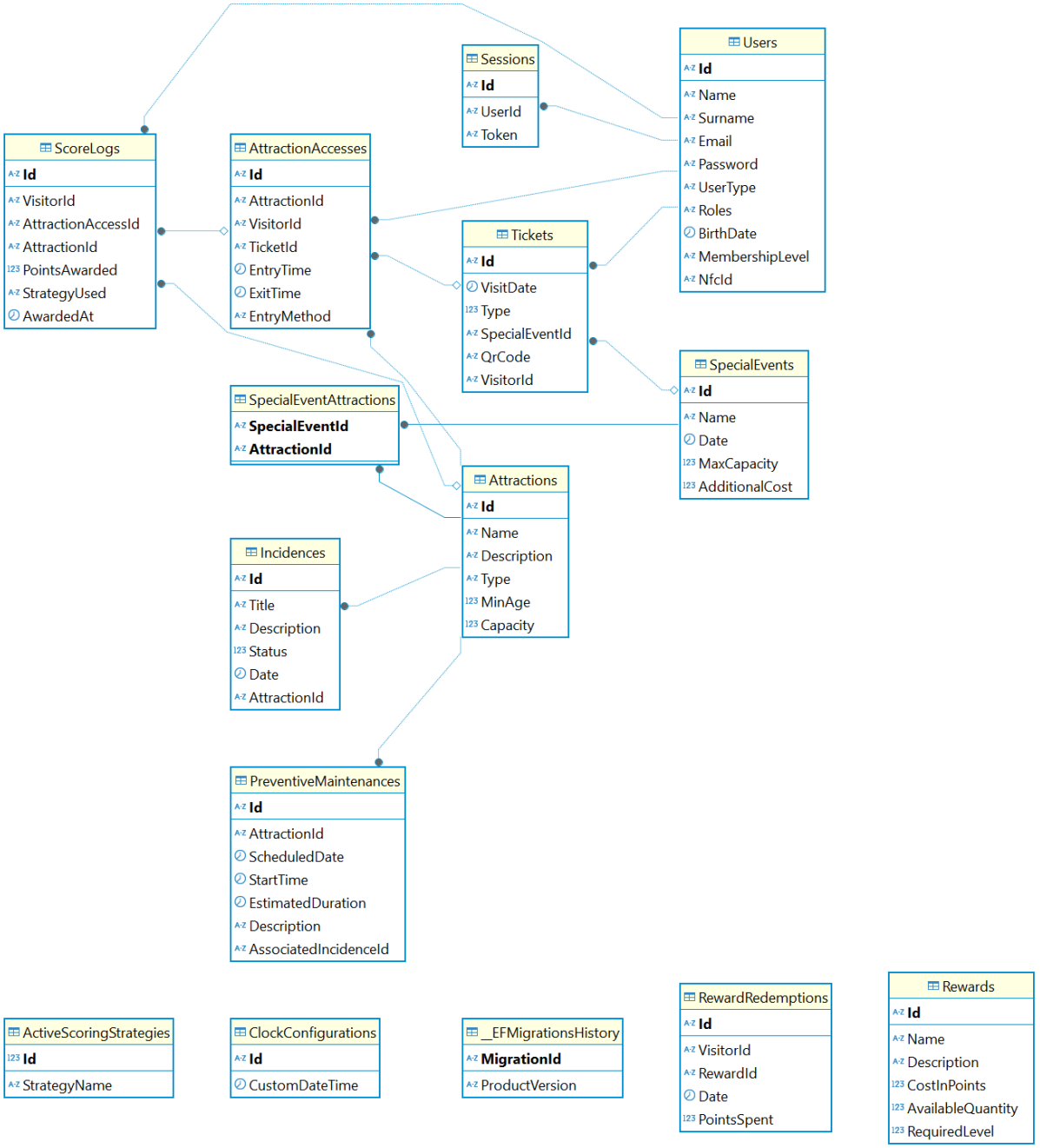
Grafo de dependencias de assemblies:



Abstractness versus Instability Diagram:



Modelo de tablas de la estructura de la base de datos



Guia de Instalacion:

1. Descargar Docker Desktop.
2. Descargar el docker compose.
3. Instalar la imagen de docker con el comando : docker compose up -d en la consola de comandos.
4. Correr el contenedor con el botón de play.
5. Descargar DBeaver.
6. Abrir DBeaver y configurar una nueva conexión a una base de datos;
7. Setear localhost, puerto 1443, usuario : “sa”, y contraseña la que está en el archivo .yaml, en nuestro caso “Passw1rd”.

8. Conectar.
9. Cargar el script con los datos de prueba.
10. Correr el ejecutable en la carpeta publish.
11. Correr el ejecutable de Angular
12. Acceder a la dirección proporcionada.
13. Loguearse con el usuario: admin@virtualpark.com , password:string

Demo:
Logearse con admin
moverse por la navbar mostrando que puede acceder a todo
Crear un usuario operador
Desloguearse
Intentar bypassear los endpoints
registrar un nuevo usuario
ver las cosas por visitor
desloguearse